GitHub

Git è uno strumento che permette di gestire il lavoro in team, si basa su versioning e collaboration.

Cosa vuol dire controllare le versioni? Vuol dire che per ogni file del mio progetto, inserisco in qualche modo delle informazioni per capire a che momento sono dello sviluppo.
Questo si fa per tener **traccia** di ogni modifica che io (o altri) faccio al mio progetto. Gli sviluppatori possono "tornare indietro nel tempo" e confrontare le vecchie versioni. E poi ogni sviluppatore può effettuare le sue modifiche in parti diverse del progetto
Una delle cose più importanti è poter **condividere** con gli altri membri del team, da remoto, quello che sto facendo in tempo reale.

Alle varie versioni posso dare dei nomi, o con le classiche 1.0, 2.0 eccetera oppure anche con dei veri e propri nomi decisi dal team (test, final, etc)
Con questi tipi di sistema posso vedere chi ha aggiunto ogni singola riga di codice e posso monitorare con degli strumenti di analytics il lavoro svolto.
Questo sistema mi permette di eliminare facilmente dei cambiamenti che ho fatto in maniera erronea, e ci permette di lavorare in maniera "rilassata" senza paura di rovinare tutto.
Ci sono delle piccole descrizioni di cosa è stato cambiato

Potrebbe succedere che due persone stiano modificando contemporaneamente la stessa linea di codice, creando dei conflitti, ma esistono dei tools fatti apposta per riuscire a gestire anche queste situazioni

Stage

Il comando che si utilizza per inizializzare una cartella di tipo git, per poterla versionare e cominciare ad utilizzare, è:

```
git init
```

Lo devo usare all'interno della cartella che voglio far diventare di forma git e la cartella verrà "trasformata" .
Verrà così creato un file di tipo git (solitamente è invisibile al file system, lo si può vedere con il comando ls -la), che conterrà tutte le informazioni necessarie per poterlo versionare e condividere. Vedremo poi che questo file possiamo modificarlo in base alle nostre esigenze per dare dei permessi particolari.

Dentro il file .git ci sono delle informazioni nascoste quindi, alcune informazioni utili che possiamo aggiungere sono nome, email, username e configurare un editor di default:

```
git config --global user.name "<your name>"
git config --global user.email "<your email>"
git config --global credential.username "<your username>"
git config --global core.editor pico
```

Se al posto di $--global$, usiamo $--local$, le credenziali e le configurazioni che faremo saranno valide solo per la cartella corrente.

Con il comando

$$git\ \ status$$

mi vengono date una serie di informazioni come ad esempio in quale branch sono, la situazione dei file, se bisogna fare una commit, se “l’albero di lavoro” è pulito etc

Per aggiungere i file da tracciare allo stage (un area “di mezzo”) usiamo:

$$git\ \ add\ \ .$$

al posto del punto posso mettere il nome del singolo file / singoli file (ad esempio per aggiungere tutti i file che sono txt usiamo $git\ \ add\ \ *.txt$), altrimenti col punto viene effettuato l’add allo stage per tutti i file non ancora tracciati, e invece usiamo:

$$git\ \ restore\ \ --staged\ \ <nomeFile>$$

per togliere i file dallo stage.
Se dopo aver effettuato una modifica di un file che abbiamo precedentemente aggiunto allo stage vogliamo riportarlo allo stato in cui era prima della modifica (ovvero ripristinare il file in base allo stage) usiamo:

$$git\ \ restore\ \ <nomeFile>$$

Ignore

Con il file chiamato .gitignore (nel top level della repo) possiamo definire i file che vogliamo che vengano ignorati da git.
Ogni linea di questo file specifica un pattern.
All’interno dobbiamo scrivere i nomi dei file che vogliamo che vengano ignorati.
Se vogliamo specificare tutti i file con la stessa estensione dobbiamo scrivere *.estensione.
* viene usato come wildcard
Per indicare delle eccezioni dall’esclusione precedente, dobbiamo scrivere !nomefile.estensione
Per escludere cartelle (non vuote, altrimenti vengono ignorate a prescindere) bisogna semplicemente scrivere il nome della cartella. Nel caso di cartelle e file interni dobbiamo specificare il percorso delle sottocartelle/file.
Possiamo (e dobbiamo per fare ordine) inserire dei commenti con # questo è un commento

Commit

Per effettuare il commit (applicare le modifiche) usiamo:

$$git\ \ commit\ \ -m\ \ "first\ \ commit"$$

con $-m\ \ "first\ \ commit"$ che va ad indicare il messaggio del commit, aggiungendo un altro parametro con $-m$ vado ad aggiungere una descrizione per il commit.
Nel caso in cui non specificassimo il messaggio, si aprirà un’interfaccia per scriverlo:

- premendo il tasto i, entreremo in modalità “insert” e potremmo modificare il messaggio
- premendo il tasto esc, chiudiamo la modalità “insert”

- digitando :*wq* e poi *enter*, finiremo le modifiche è verrà fatta la commit con il messaggio da noi scritto (il significato è : per entrare in command mode, w per salvare (write) e q per uscire (quit).
  Oppure possiamo digitare :*x* (save and exit)
- se non vogliamo aggiungere nessun messaggio basta digitare :*q* e poi *enter*

Per eseguire add e commit insieme possiamo usare il comando:

$$git\ \ commit\ \ -am$$

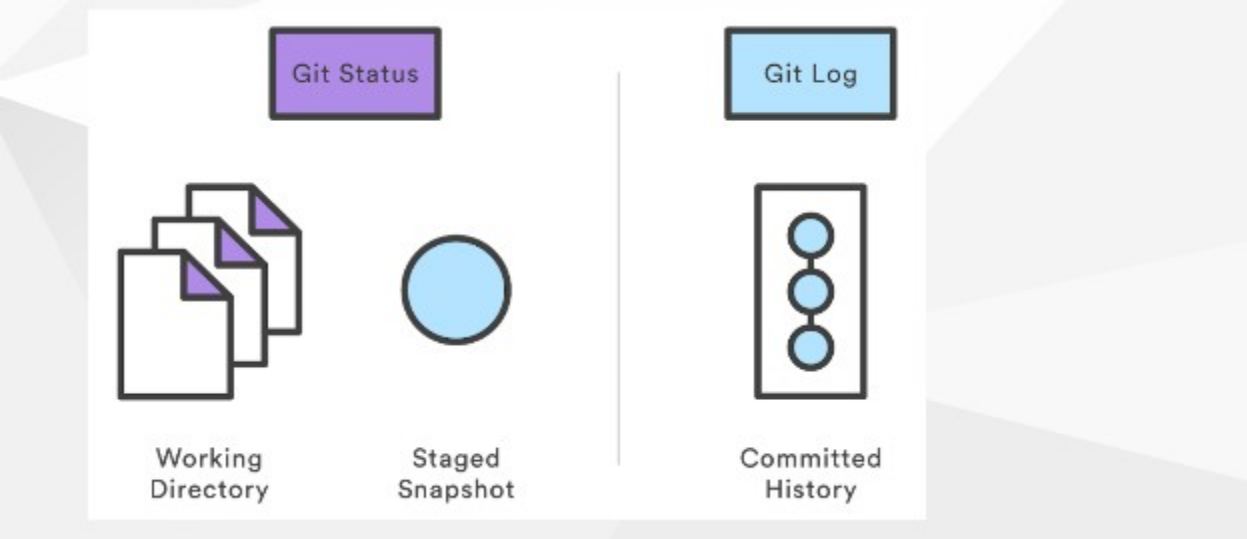
però non verrà effettuata la add per i nuovi file ma solo per quelli già esistenti e modificati.
Esiste anche un comando per modificare l'ultima commit eseguita (magari quando sbagliamo il messaggio):

$$git\ \ commit\ \ -amend$$

Per controllare la storia delle commit usiamo:

$$git\ \ \log$$

e possiamo usare --*oneline* (per ogni commit su una riga) e/oppure --*reverse* (per visualizzare i commit in ordine di meno recente)



Possiamo vedere le differenze di un file in base a delle commit precedenti in due modi, o indicando quante commit precedenti rispetto al puntatore HEAD

$$git\ \ diff\ \ master\,\hat{}\,\hat{}\ \ index.js$$

in questo caso 3 commit (si parte da 1 e per ogni ^ si aggiunge una commit), oppure specificando il codice della commit (visualizzabile dal log)

$$git\ \ diff\ \ <commit\ \ ID>\ \ index.js$$

Possiamo ripristinare una commit precedente (vedendo il codice della commit da un log) con il comando:

$$git\ \ reset\ \ <ID\ \ commit>$$

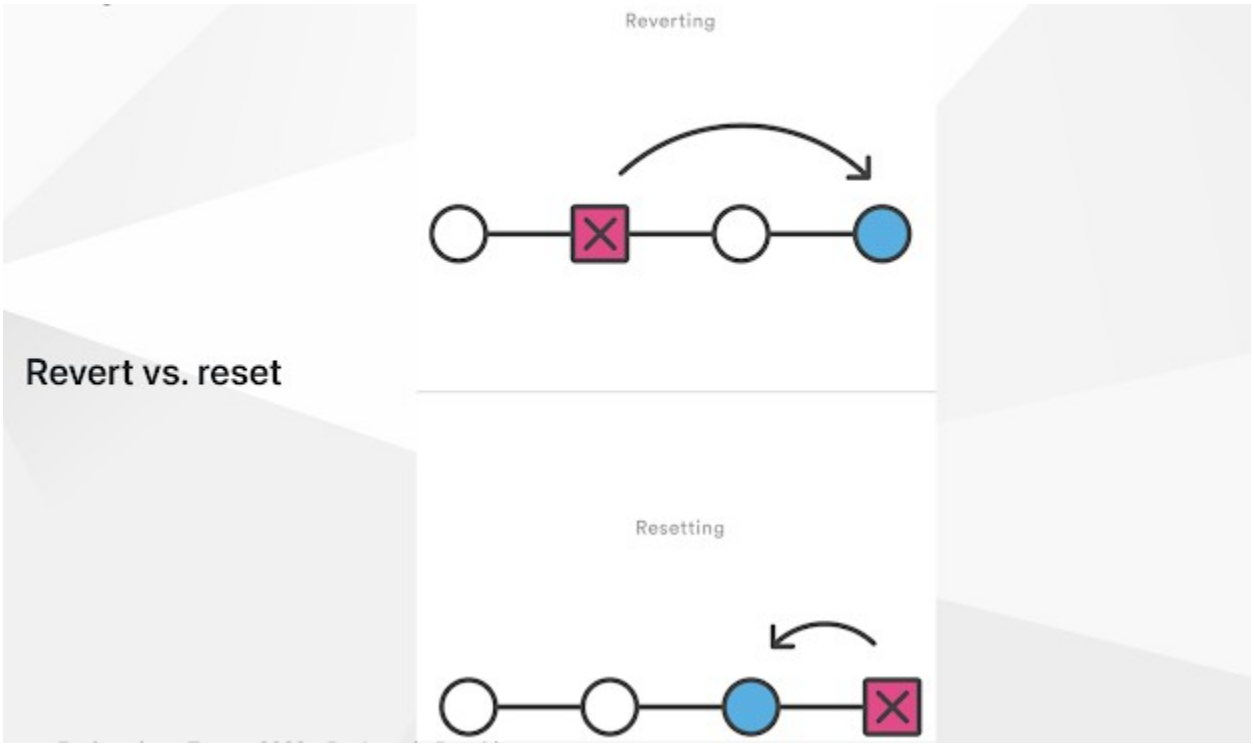
così facendo però cancelliamo definitivamente tutte le commit successive (per evitare questo si usa git reverse). Esistono tre versioni per la git reset:

- --*soft*
  Ci riporta alla penultima commit, però avendo la commit appena cancellata nello stage

- $--mixed$ (default)
  È come se si facesse una $git\ \ restore\ \ --staged$, quindi annulliamo la commit ma abbiamo ancora i file nella nostra working directory
- $--hard$
  Andiamo a ripristinare completamente allo stato precedente la working directory

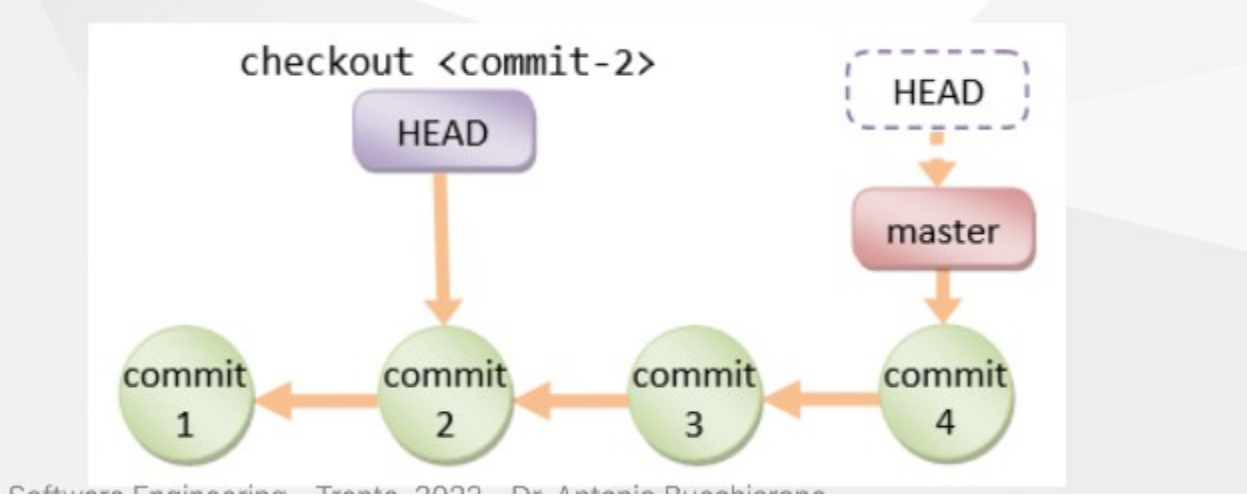
Se non vogliamo specificare l'ID COMMIT possiamo muoverci dall'HEAD scrivendo $HEAD \sim n$ dove n è il numero di commit indietro rispetto al puntatore $HEAD$ (si può usare git reset HEAD per rimuovere tutti i file dalla staging area)
Esiste anche una specie di undo, che però non è che elimina definitivamente l'ultimo commit, ma ne aggiunge un altro senza le modifiche dell'ultimo commit, così da mantenere la cronologia:

$git\ \ revert$



Il puntatore HEAD punta al branch su cui stiamo lavorando (di default master). Il branch a cui stiamo lavorando punta a sua volta all'ultima commit effettuata su quel branch.

Il comando checkout serve per spostare il puntatore HEAD, e si può usare sia per spostarsi su commit precedenti sia per cambiare branch.



Branching

I **branch** sono il modo migliore per separare il lavoro del progetto in più parti quando si lavora in team. Una volta che il lavoro in un branch viene finito ed è funzionante, si può aggiungere/applicare al branch master con un merge.

Con il comando

git branch

possiamo visualizzare i branch e vedere su quale stiamo lavorando.
Per creare un nuovo branch usiamo

git branch newFunction

dandogli un nome, in questo caso “newFunction”

Per poter cambiare branch devo fare un checkout, oppure uso il comando che fa entrambe le cose, creazione e checkout:

git checkout -b newFunction

Se noi creiamo un nuovo file in un branch, e poi torniamo al branch master, vedremo che il file non ci sarà più. Questo perché quel file esiste solo sull’altro branch e non sul master.

Per cancellare un branch uso il comando (delete):

git branch -d newFunction

Per poter vedere le differenze tra due branch usiamo il comando:

git diff master..branch

Per fare un merge usiamo:

git merge newFunction

di solito si usa dopo aver fatto il checkout nel master, così da avere sempre tutto nel master.

Se il branch su cui andiamo a eseguire il merge è un “parente diretto” di master, non avremo problemi per il merge, altrimenti avremo dei conflitti che dovremo risolvere (un tool grafico per risolvere questi problemi è git config merge.tool p4merge).

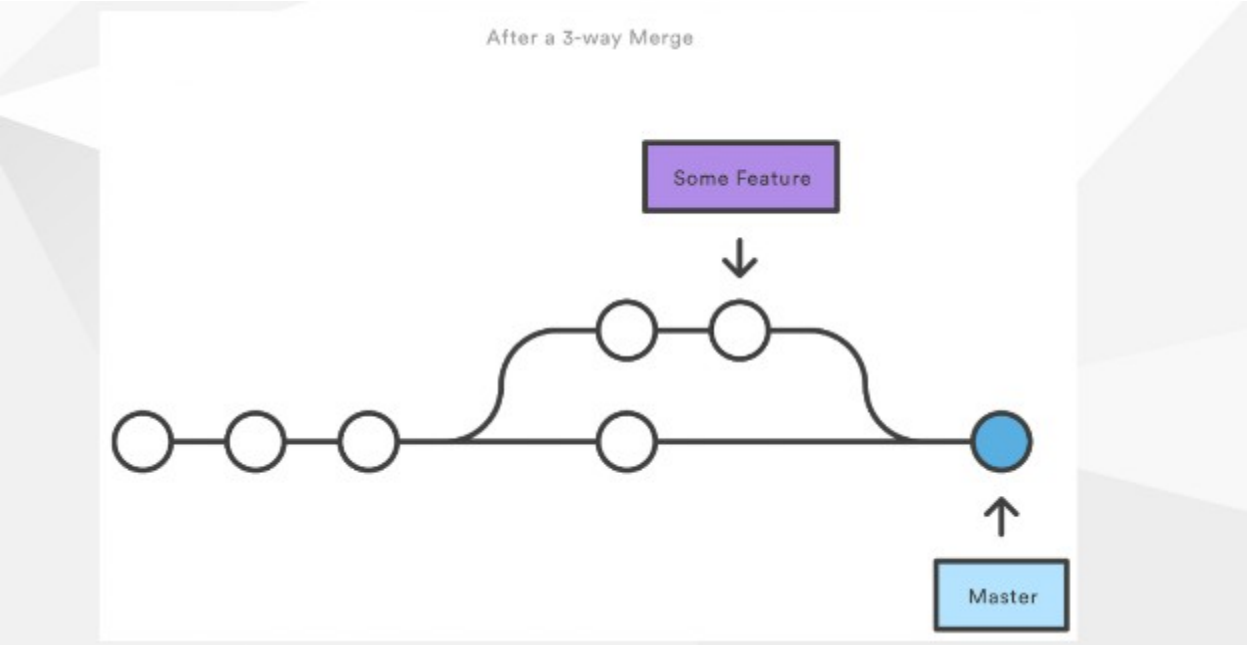
Posso vedere da terminale la storia dei branch e delle sezioni di sviluppo con il comando:

$$git\ \log\ --graph\ --decorate\ --oneline\ --all$$

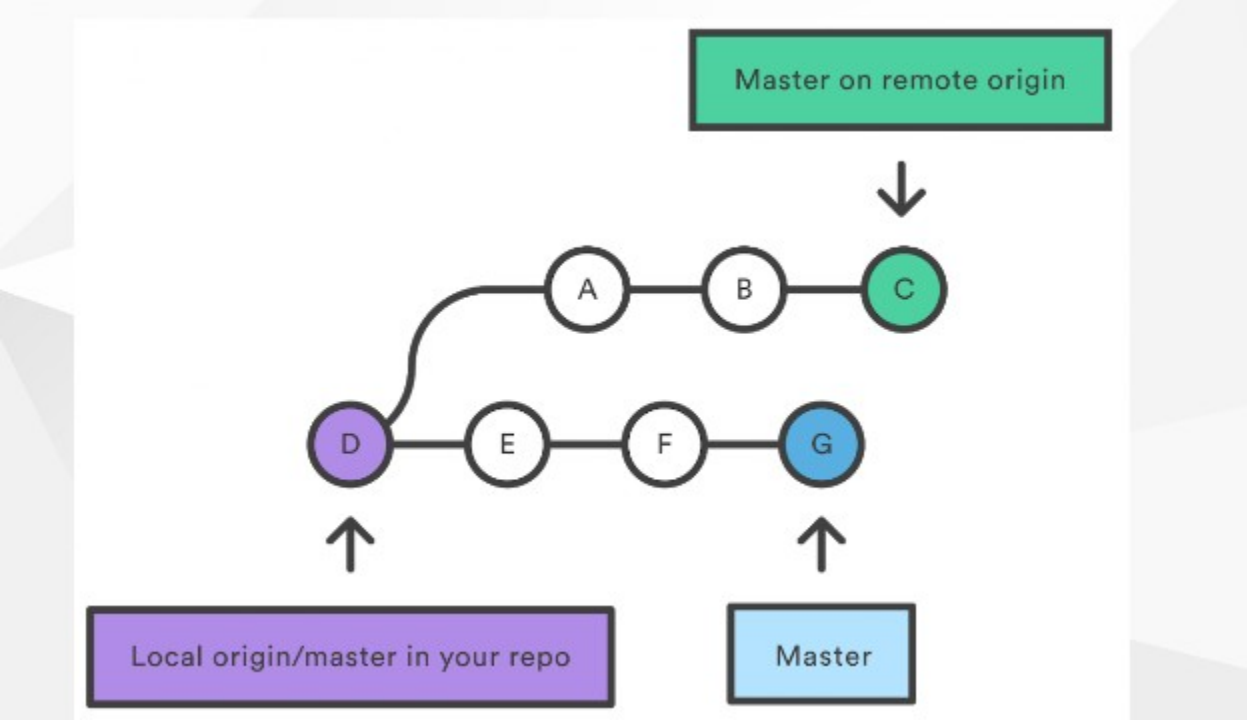
Con il comando:

$$git\ branch\ --merged$$

andiamo a vedere quali branch sono stati uniti (così da poter eliminare i branch inutili)



Come abbiamo detto è tutto pensato per facilitare la collaborazione e il lavoro in team, quindi io avrò il mio master in locale ma ci sarà un master in remoto, che sarà più “avanti” o più “indietro” del mio



GitHub

Per collegare una cartella di GitHub con la nostra in locale usiamo il comando:

$$git\ \ remote\ \ add\ \ origin\ \ <urlCartella>$$

Così colleghiamo il nome origin all'url (possiamo anche usare nomi diversi). Possiamo visualizzare tutti gli url collegati con il comando:

$$git\ \ remote\ \ -v$$

Per cancellare un server remoto usiamo:

$$git\ \ remote\ \ remove\ \ origin$$

Per copiare una cartella di GitHub sulla nostra macchina in locale (sempre se si possiedono i permessi) si fa con:

$$git\ \ clone\ \ [URL]$$

in questo caso verrà settato in automatico origin, se vogliamo cambiare nome alla cartella che andiamo a copiare, basta scriverlo dopo l'URL.
Possiamo controllare le differenze, una volta modificato un file, con la cartella remota:

$$git\ \ diff\ \ origin\ \ master\ \ index.js$$

Per riprendere il codice da remoto, sovrascrivendo qualsiasi cosa io abbia fatto in locale, possiamo usare due metodi:

$$git\ \ fetch$$
$$git\ \ merge\ \ origin/master$$

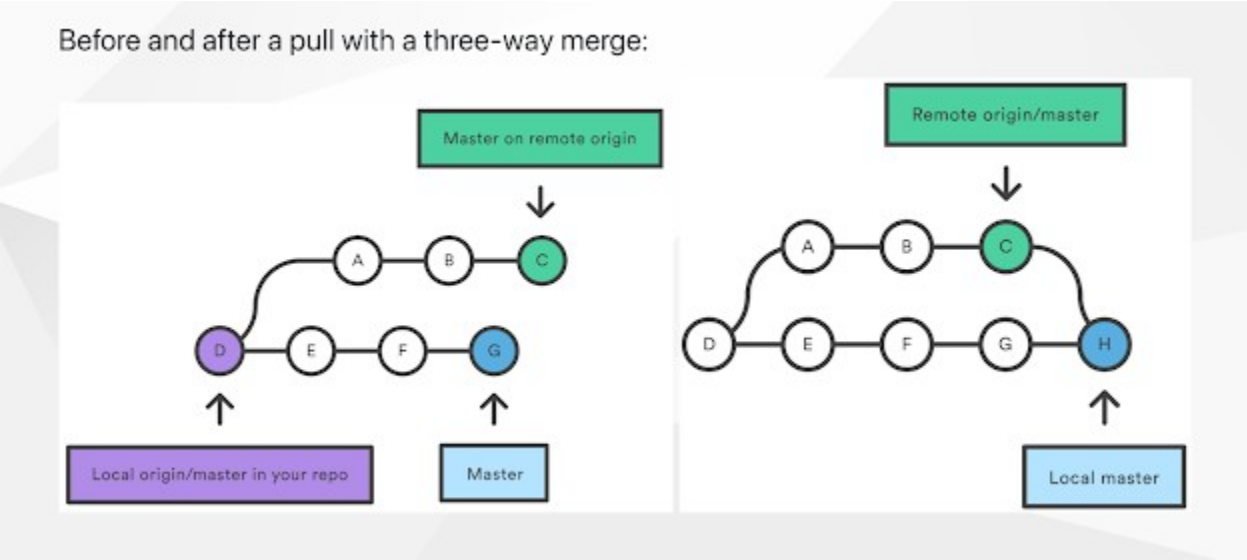
oppure

$$git\ \ pull$$

per unire i due comandi.

Con il push invece, quando voglio che il codice che ho fatto venga salvato in remoto, posso caricarlo nello specifico branch

$$git\ \ push\ \ origin\ \ master$$



Per fare un git status in remoto usiamo:

$$git\ \ remote\ \ -v$$

Per aggiungere un file al git remoto usiamo:

$$git\ \ remote\ \ add\ \ <nome\ \ file><url>$$

Per fare la delete di un branch al git remoto usiamo:

$$git\ \ branch\ \ -d\ [branch_name]$$

$$git\ \ push\ \ origin\ \ -d\ [branch_name]$$

$$git\ \ push\ \ origin\ :[branch_name]$$

Quando abbiamo un lavoro lineare, e vogliamo che un branch secondario diventi “ufficiale”, quindi master, usiamo il comando:

$$git\ \ rebase\ \ master$$

Per questo comando ci sono varie versioni:



Esistono i tag, che possono essere di “basso peso” ($git\ \ tag\ \ v1.0$) oppure annotati, ovvero con anche informazioni tipo il nome di chi esegue il tag, l’email e la data ($git\ \ tag\ \ -a\ \ v1.4$).
Si può pensare ai tag di basso peso come privati e ai tag annotati come pubblici. Con l’estensione $-m$ si può aggiungere un messaggio.
Per visualizzare tutti i tag in una repo usiamo: $git\ \ tag$
I tag a differenza delle branch sono statici, e una volta creati restano li

MongoDB e Mongoose

JSON

MongoDB utilizza un formato di gestione dei dati chiamato JSON (ovvero JavaScript Object Notation). JSON è una formattazione del testo usata per salvare e trasferire dati.
JSON è auto-descrittivo e facile da capire.
Vengono usate le parentesi graffe per gli oggetti e le parentesi quadre per gli array
I valori in JSON devono essere dei seguenti tipi:

- a string
- a number
- an object
- an array
- a boolean
- null



In JavaScript c’è la possibilità di elaborare i dati scritti in JSON, infatti ci sono una serie di funzioni che ci aiutano a gestirli.
Con JSON.parse() mi viene ritornato un oggetto javascript in base alla stringa JSON che passo come parametro.

Con il comando $document.getElementById("demo1").innerHTML$ noi andiamo a selezionare l'elemento del documento con ID uguale a $demo1$ e andiamo a sostituirlo con il testo che scriveremo dopo l'uguale
$innerHTML$ = inserire all'interno di quel tag una stringa (va a sostituire nel caso in cui ci sia già testo)

Javascript è capace di trasformare una stringa formattata in una certa maniera in un oggetto di tipo data usando $new\ \ Date(obj)$

Si possono formare delle strutture composte avendo oggetti dentro array e/o viceversa

NodeJS

NodeJS è un runtime system open source multipiattaforma orientato agli eventi per l'esecuzione di codice JavaScript.
Per vedere se si ha installato Node e che versione si usa da terminale:

$$node\ \ -v$$

È basato su JavaScript (interpreta codice javascript, e si basa sulla versione 8 di JavaScript).
Ha come browser di riferimento Google Chrome.
Quando Node.js sta eseguendo operazioni di input/output (leggere dalla rete, accedere a un database etc), invece di bloccare il thread e sprecare cicli di CPU, NodeJS riprende l'operazione una volta che la risposta è arrivata.
Questo permette a NodeJS di gestire mille connessioni concorrenti con un singolo server senza introdurre il peso di gestire la concorrenza di thread.

npm è un gestore che fornisce tutte le librerie che possiamo utilizzare in nodejs.
Per vedere se si ha installato npm e che versione si usa da terminale:

$$npm\ \ -v$$

Altrimenti si installa con il comando:

$$npm\ \ install\ \ -g\ \ npm$$

Per installarlo bisogna dargli i permessi dell'amministratore

Possiamo creare un server in locale, con require chiediamo di usare una libreria chiamata 'http' (un package di npm), e la associamo ad un oggetto (chiamato anch'esso 'http').

```
const http = require('http');

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'text/plain');
    res.end('Hello World');
});

server.listen(port, hostname, () => {
    console.log(`Server running at http://${hostname}:${port}/`);
});
```

Facendo partire questo codice con il comando *node server.js* creeremo un server in locale, che possiamo raggiungere cercando sul nostro browser http://127.0.0.1:3000/ , e ci mostrerà a video la scritta 'Hello World'.

Per installare libreria di npm bisogna andare sul sito di npm e cercare le librerie per vedere a cosa servono e come si installano.
console.log() permette di scrivere dei log nel nostro codice (delle stampe a terminale)

MongoDB

MongoDB salva i dati in un documento JSON flessibile, ovvero che i campi possono cambiare da documento a documento e la struttura dati può cambiare nel tempo.
Il modello del documento "prende forma" dagli oggetti del codice della nostra applicazione, e questo rende molto semplice l'interfacciamento.
La cluster è la macchina virtuale che gestirà il db.

Come prima cosa dobbiamo creare un progetto.
Successivamente dobbiamo creare i **database**, scegliendo username e password da usare per connetterci.
Poi dobbiamo dare i permessi di accesso al nostro indirizzo IP (visto che non abbiamo un IP statico facciamo prima a mettere i permessi a tutti gli indirizzi) (Network access -> IP address -> allow access from anywhere)
Successivamente clicchiamo su connect sul database e scegliamo "MongoDB drivers", poi scegliamo Node.js e ci farà copiare e incollare il codice javascript da incollare per effettuare la connessione al database, rimpiazzando <password> con la nostra password

Mongoose

Mongoose è una libreria di NodeJS che ci aiuta a modellare e gestire oggetti mongodb in javascript.

Mongoose mette a disposizione dei metodi semplici per modellare i dati della tua applicazione, inclusi il casting di tipi integrato, la convalida, la costruzione di query, logica di business e molto altro.
Per installare la libreria di Mongoose bisogna usare il comando:

```
npm install mongoose
```

Successivamente nel nostro file che userà mongoose dovremo inserire la riga di codice seguente:

```
const mongoose = require("mongoose");
```

Successivamente possiamo creare il nostro **schema** di database, ad esempio:



Successivamente per poter usare lo schema, dobbiamo convertirlo in un **modello**, con cui possiamo lavorare:

```
const Student = mongoose.model('Student', studendSchema);

//new student added to the DB
const student = new Student({ name: 'Antonio', surname: 'Bucchiarone' });
student.save(function (err) {
    if (err) return handleError(err);
    // saved!
    console.log("Student added");
});
```

Esiste una funzione per inserire più oggetti (tramite array):

```
Student.insertMany(
    [
        { name: 'mario', surname: 'rossi' },
        { name: 'paolo', surname: 'neri' }],
    function (err) {
        console.log("array of students added");
    });
```

Esistono anche le funzioni per eliminare uno o più elementi in base ad una caratteristica:

- Models have static `deleteOne()` and `deleteMany()` functions for removing all documents matching the given filter.

```
Student.deleteOne({ name: 'Antonio' }, function (err) {
  if (err) return handleError(err);
  // deleted at most one tank document
  console.log("element deleted");
});
```

```
Student.deleteMany({ name: 'Antonio' }, function (err) {
    if (err) return handleError(err);
    // deleted at most one tank document
    console.log("elements deleted");
});
```

Esistono vari metodi per eseguire una query di ricerca (una SELECT), ecco alcuni esempi:

Finding documents is easy with Mongoose, which supports the rich query syntax of MongoDB. Documents can be retrieved using a model's **find**, **findById**, **findOne**, or **where** static methods.

```
// Find one student whose `name` is 'Antonio', otherwise `null`
Student.findOne({ name: 'Antonio' }).exec();

// using callback
Student.findOne({ name: 'Antonio' }, function (err, student) {
    console.log("surname: "+student.surname);
});

// select only the students name and surname
Student.findOne({ name: 'Antonio' }, 'name surname').exec();
```

Subdocuments versus Nested Paths

> https://mongoosejs.com/docs/subdocs.html#subdocuments-versus-nested-paths

```
// Subdocument
const subdocumentSchema = new mongoose.Schema({
  child: new mongoose.Schema({ name: String, age: Number })
});
const Subdoc = mongoose.model('Subdoc', subdocumentSchema);

// Nested path
const nestedSchema = new mongoose.Schema({
  child: { name: String, age: Number }
});
const Nested = mongoose.model('Nested', nestedSchema);
```

BackEnd

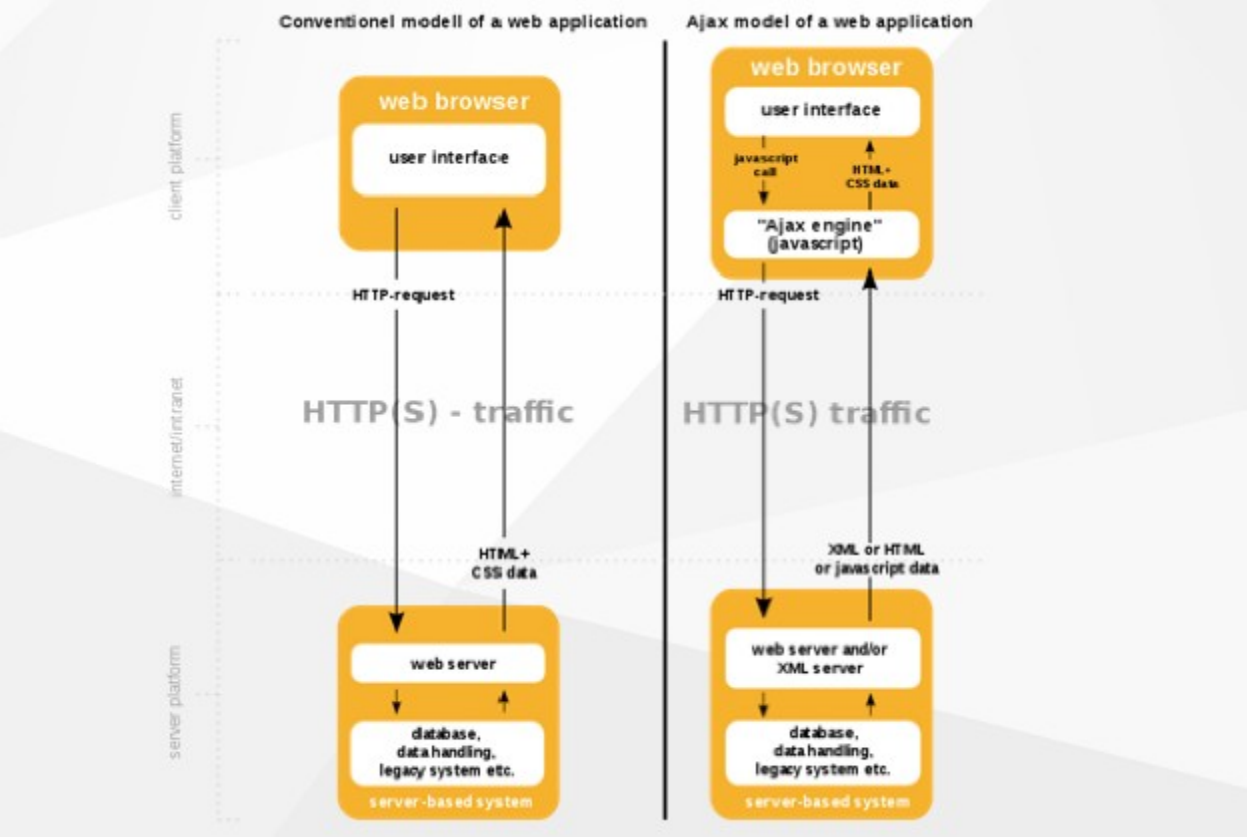
Il **Web 1.0** è associabile alla prima fase di sviluppo del web, in cui pagine statiche collegate da hyperlink compongono la maggior parte dei contenuti fruibili sul web.
Con lo sviluppo delle tecnologie e le crescenti richieste da parte degli utenti, si è sviluppato il **Web 2.0**, dove i contenuti vengono fruiti come **servizi** (nascono quindi le **Web Application**).
I vantaggi delle tecnologie web sono:

- **Disponibilità e facilità di accesso**
  Basta una connessione internet, un dispositivo (mobile o desktop) e un browser
  Si può anche simulare un server e fare tutto sulla stessa macchina senza fare il deploy online del sito
- **Condivisione**
  Tanti utenti, stessi dati (le pagine solitamente sono singole ma vengono fruite a migliaia di utenti)
- **Tecnologia standard**
  Permette uno sviluppo semplice

Il Web 2.0 ha portato una serie di tecnologie molto utili, grazie soprattutto ad **AJAX** (Asynchronous JavaScript and XML) che è una tecnologia che definisce un **pattern standard** per la comunicazione tra back-end e front-end di applicazioni web.
Altre tecnologie che caratterizzano e formano il web 2.0 e AJAX sono:

- **HTML e CSS**
  Definizione di grafica e posizionamento componenti di una pagina web **statica**
- **Javascript e DOM**
  Permettono di aggiungere dinamicità alle pagine web
  DOM = Dynamic Object Management, mi rende facile l'accesso ad oggetti HTML
  Grazie al DOM, è possibile interagire con gli elementi di una pagina html e modificarli in tempo reale, ad esempio come reazione ad input dell'utente.
- **XMLHttpRequest**
  È un API supportata dai "moderni" browser che permette alla pagina web di comunicare con un backend o con servizi terzi.
  Specificatamente è possibile comunicare con un server web dall'interno di una pagina.
- **Backend**
  Composto da linguaggi di scripting (come PHP) e veri e propri linguaggi di programmazione (Servlet Java, Node.js, etc)
- **Web API e servizi RESTful**
  Per scomporre l'applicazione in più componenti e nodi

Ecco la differenza dell'architettura tra un'applicazione Web 1.0 e una Web 2.0



Quando si vuole fare un'applicazione web responsive (cioè che vada bene per tutti e su tutti i dispositivi) queste sono le tecnologie che vengono usate più spesso ad oggi:



Javascript è un linguaggio “leggero”, **interpretato** (oppure compilato just-in-time) con **funzioni di prima classe**, ma è conosciuto come il **linguaggio di script per il web**
Javascript è basato sui prototipi, è multi paradigma, è single-threaded, è un linguaggio dinamico, supporta la programmazione ad oggetti, è imperativo e dichiarativo.

Node.js è una piattaforma lato server. Node.js è guidato dagli eventi, è single threaded ed è perfetto per le applicazioni real time con tanti scambi di dati che girano su sistemi diversi.

Il software che ci servirà per programmare la nostra applicazione sarà:

- Un Web Browser
- Node.js (con npm)
- Un editor (come Visual Studio Code)
- Postman (tool che permette di verificare il nostro codice e soprattutto le nostre API)
- apiary.io
- Git CLI
- github.com

JavaScript

Per fare un output sul terminale su JavaScript usiamo $console.\log(stringa)$ (utile per fare **debug**).
Per runnare un file JavaScript usiamo $node\ \ nomeFile.js$

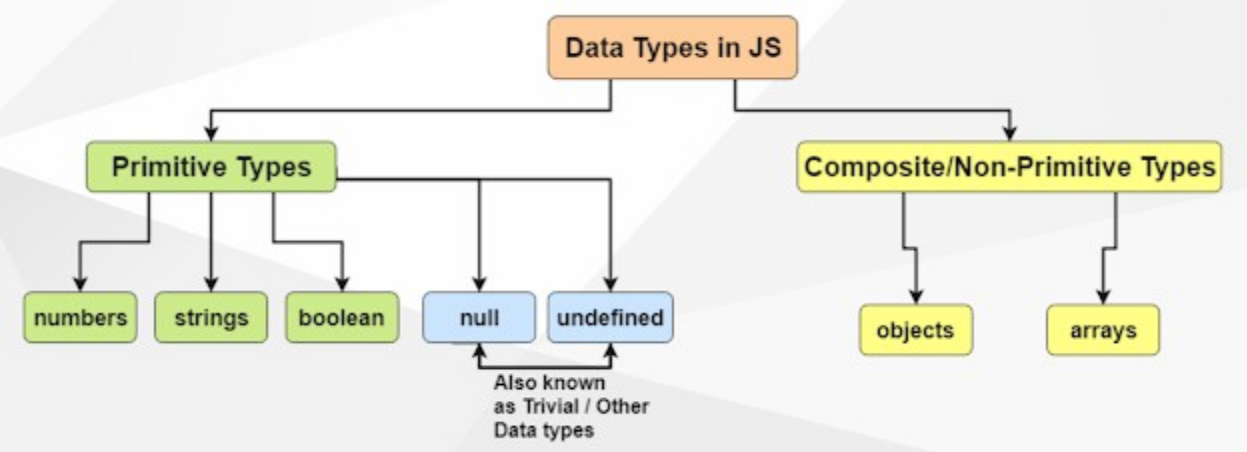
Come in un qualsiasi linguaggio di programmazione posso dichiarare **variabili** e **costanti** (senza tipi, visto che la tipizzazione viene fatta a run time), **funzioni** e **costrutti di controllo del flusso**:



```
// declaration of variables - JavaScript is an untyped language!
const a;  var b; let c;

// declaration of a function (first-class citizens)
function sum(a,b) {}
var result = sum(a, b);

// control flow statements
if (true) {} else if () {} else {}
for (let i=0; i<10; i++) {}
while (true) {}
...
```



undefined si usa per gestire le eccezioni

Nel seguente esempio vediamo che una variabile che non è ancora stata utilizzata è di tipo undefined, invece tutte le altre vengono usate e quindi assumeranno un tipo.
Con l'operatore *typeof* () si può vedere di che tipo è una variabile:

```
var myvar;
console.log(typeof (myvar));            // undefined

myvar = 'Pippo';                        // string
myvar = 5;                              // number
myvar = true;                           // boolean
myvar = [1,2,3];                        // object
myvar = {key1: "value1"};               // object
myvar = null;                           // object
myvar = function(n){return n+1};        // function

console.log(Array.isArray([1,2,3]));    // true
```

Notiamo che il tipo di una variabile può cambiare nel corso di un programma.
Per gli array le operazioni sono le seguenti:

```
var list = ["apple", "pear", "peach"];  //list of elements
list[0]                                 // accessing an element by id
list.indexOf("pear")                    // checking the index of "pear" in the array
list.push("banana");                    // Adding a new element
list.pop()                              // Taking the last element from the array
list.shift()                            // Taking the first element
list.length                             // checking the number of elements
list.slice(start, end)                  // copy a subportion of the original array
list.join('separator')                  // return string by concatening elements
```

Un altro aspetto importante è lo **scope delle variabili**:

global scope

```
const me = 'me';    // outside of any function or block
```

function scope

```
function myF () { const me = 'me'; }
console.log(me)     // ReferenceError
```

block scope

```
const year = 2023;
if (year > 2022) { const me = 'me'; var you = 'you';
    console.log(you)   // you
    console.log(me)
}
```

Ovviamente abbiamo anche gli operatori logici:

- == equality
- *!* = not equality
- === strict equality
- *!* == strict inequality

```
// equality
2 == 2    // true
2 == '2'  // true

// inequality
2 != 2    // false
2 != '2'  // false

// strict equality
2 === 2   // true
2 === '2' // false

// strict inequality
2 !== 2   // false
2 !== '2' // true
```

Gli operatori matematici:

- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to

I costrutti di iterazione:

- Ciclo while
- Ciclo for
- Ciclo for each sugli array

```
while (condition) { console.log('do') }
```

```
for (var i=0; i< 100; i++) {
    if ((i%2)==0) continue;     // if even, skip to the next cycle
    console.log(i);             // else, print i
    if (i>=10) break;           // when greater equal then 10, quit the loop
}
```

```
for (let value of ['first','second']) {
    console.log(value)          // value is the item in the array
}
```

```
[1,2,3].forEach( console.log )      // callback receives the item as parameter

[1,2,3].forEach( v=>console.log(v) )// callback defined as an arrow function
```

Le funzioni:

How to define a function

```
function add(a, b) {
    return a + b;
};

var mult = function (a, b) { return a * b * this; };

var arrowFunction = (a,b) => a * b;
```

How to call a function

```
add(1,2)            //3
mult.bind(2)(2,2)   //8 - bind() specifies a value for the keyword this.
```

Possiamo definire un oggetto senza definirne la classe:

```
var car = {
    type : 'Fiat',
    model : '500',
    color : 'red',
    description : function() {
        return this.color + ", " + this.model + ", " + this.type;
    }
    // methods cannot be defined using arrow functions!
    // in the case of arrow functions, context 'this' is not assotiated to the object
}
console.log(car);
console.log(car.description());
```

Ma con le nuove versioni di JavaScript possiamo definire una classe con la keyword *class*, ovviamente devo definire come minimo il costruttore chiamato *constructor*:

```
class Car3 {
    constructor(type, model, color) {
        this.type = type;
        this.model = model;
        this.color = color;
    }
    description() {
        return this.color + ", " + this.model + ", " + this.type;
    };
}
var fiatPuntobianca = new Car3('Fiat', 'Punto', 'white');
console.log(fiatPuntobianca);
console.log(fiatPuntobianca.description());
```

Si possono fare anche le estensioni (gli extend, per fare una classe figlia).

NodeJS

In NodeJS possiamo passare dei parametri al programma che andiamo ad eseguire.
Di default il primo parametro è il comando node, il secondo è il programma e i successivi sono quelli dell'utente, i parametri vengono memorizzati nell'array $process.argv$, dove $process$ è una variabile globale che contiene informazioni riguardanti il processo corrente di NodeJS.
Ecco un esempio di lettura di parametri:

Often you need to access to command line parameters. For example,

```
$ node examples2.js <from> <to>
```

In node.js you do this by accessing *process*, which is a global variable containing informationan about the current node.js process. For example:

```
for (var i = 0; i < process.argv.length; i++) {
    console.log(i + ' -> ' + (process.argv[i]));
}
```

In JavaScript possiamo anche accedere ai file, questo si fa con un package di NodeJS chiamato *fs* (file system library, è standard non serve installarlo), ecco un esempio:

```
// Loading the file system library
var fs = require("fs");
// File name from the common line params
var fileName = process.argv[2];
// Accessing the content of the file synchnously
var data = fs.readFileSync(fileName, "utf8");
console.log(data);
```

Now try running the code

```
node examples2.js path/to/file
```

La funzione *readFileSync* è una funzione di *fs* che mi permette di leggere da un file in maniera **sincrona** (quindi devo aspettare il risultato della chiamata prima di andare avanti)

Se vogliamo una chiamata **asincrona** (ovvero che viene fatta in background senza bloccare l'esecuzione del nostro codice):

```
// Loading the file system library
var fs = require("fs");

// File name from the common line params
var fileName = process.argv[2];

// Accessing the content of the file asynchnously
fs.readFile(fileName, "utf8", function(error, data) {
  console.log(data);
});

console.log("Program ended.");
```

Questa è chiamata funzione **callback**, perchè viene chiamata alla fine del completamento del task, così da non bloccare il run del programma.

Esistono quindi i **blocking method** (sincroni) e i **non-blocking methods** (asincroni)

Chiamate asincrone

Le chiamate asincrone possono "aspettare" vari tipi di evento
Ci sono diversi modi per gestire del codice asincrono (ovvero che fa chiamate asincrone) in NodeJS e JavaScript:

- Callbacks
- Promises
- Async/Await

Una **promise** è un oggetto che può avere 3 possibili stati:

- Pending
  Lo stato iniziale, prima che l'evento succeda
- Resolved
  Dopo che l'operazione è stata eseguita correttamente
- Rejected
  Se l'operazione ha avuto errori durante l'esecuzione e/o è fallita

Una promise in NodeJS è come una promessa nella vita reale, è una garanzia che una cosa verrà fatta. Però poi può succedere che venga veramente eseguita oppure no.

```
var myPromise = new Promise(function (res) {
    // async code to be executed, when ready, call res() to resolve the promise
    res('any_value');
});
// when not yet resolved
console.log(myPromise);      //Promise { <pending> }
// once resolved
console.log(myPromise);      //Promise { 'any value' }
```

Per gestire una promise una volta effettuata abbiamo due modi (in base a se è resolved o rejected):

- Il metodo .then (resolved)
- Il metodo .catch (rejected)
- Il metodo .finally che viene eseguito una volta per fare qualcosa a prescindere dallo stato

È importante notare che le callbacks non avevano la gestione in caso di errori.

Invece di fare chiamate a qualcosa possiamo anche impostare dei timeout sia callback che promise:

Suppose we want a sequence of timeouts. With callback-based setTimeout() we have:

```
setTimeout(()=>{
    setTimeout(()=>{
            console.log(123)
        }, 5000)    // later, wait for another 5 seconds
}, 1000)    // first wait 1 second
```

After promisifing the setTimeout:

```
myPromisifiedTimeout = function (time) {
    return new Promise( (res) => setTimeout(res, time) )
}
```

```
myPromisifiedTimeout(1000)              //first wait for 1 second
.then(()=>{return myPromisifiedTimeout(5000)})   // later wait for additional 5
.then(()=>console.log(123))
```

Ecco un esempio di lettura da un file con una promise:

```
const fs = require('fs');

const readPromisify = function (file) {
    return new Promise(function (resolve, reject) {
        fs.readFile(file, (err, data) => {
            if (err) throw reject(err);
            resolve(data); data
        });
    });
}

readPromisify('/file.md')
.then(function (data) {
      console.log("Promise resolved successfully");
})
.catch(function (err) {
    console.log("Promise is rejected");
});
```


C’è per ultimo il metodo dell’**Async/Await**.
Le funzioni con *async* prima del nome della dichiarazione della funzione significano che ritorneranno un valore “promise”. Altri valori saranno wrappati automaticamente in promise.

Gli *await* funzionano solo dentro a funzioni *async* e aspettano la risoluzione di una promise per poi ritornare il risultato.

The word "async" before a function means one simple thing: a function always returns a promise. Other values are wrapped in a resolved promise automatically.

```
async function f() { return 1; }
f().then(alert); // 1
```

Await works only inside async functions. It waits for a promise to resolve, then return resolved value.

```
async function f() {
  let result = await promise; // wait until the promise resolves (*)
  alert(result); // "done!"
}
```

Ecco altri esempi di Async/Await:

```
function example() {
    myPromisifiedTimeout(1000)              //first wait for 1 second
    .then(()=>{return myPromisifiedTimeout(5000)})   // later wait for additional 5
    .then(()=>console.log(123);)
}
example();
```

```
async function asyncExample() {
    await myPromisifiedTimeout(1000);   //first wait for 1 second
    await myPromisifiedTimeout(5000);   // later wait for additional 5
    console.log(123);
}
asyncExample();
```

Librerie

Con l'installazione di Node.js vengono anche installate alcune librerie standard (come *fs* per l'accesso al file system).
Per poter usare questo modulo all'interno del nostro programma dobbiamo usare

$$var\ \ fs\ =\ require("fs");$$

Questa istruzione carica il modulo *fs* e ne assegna una sua istanza alla variabile *fs*. Attraverso questa istanza possiamo avere accesso ad ogni funzionalità esportata da quel modulo.

Possiamo anche creare i nostri moduli e poi usarli in altri nostri file:

./user.js

```
function userTemplate(user) {
  return `Name: ${user.name}`;
}
module.exports = userTemplate;
```

./index.js

```
const userTemplate = require('./user');
console.log( userTemplate({name:'marco'}) );
```

NPM è un potente strumento che aiuta molto nella gestione della dipendenza dei progetti e soprattutto aiuta ad automatizzare i flussi di lavoro di gruppo.

Il file *package.json* contiene i metadati riguardanti il progetto (come ad esempio nome, autore, versione, licenze e dipendenze). Nonostante si possano installare dipendenze anche senza un file *package.json*, usarlo è una delle migliori pratiche per tener traccia delle dipendenze.

Per iniziare ad utilizzare npm dobbiamo inizializzare il progetto con il comando

$$npm\ \ init$$

Questo comando genererà il file *package.json*

Se vogliamo installare un modulo esterno (non incluso nei moduli standard di Node.js) usiamo il comando

npm install nomePacchetto

E con il parametro *--save* indichiamo a *npm* di aggiungere il modulo alla lista delle dipendenze nel file *package.json*

Per disinstallare un pacchetto usiamo

npm uninstall nomePacchetto

Ecco un esempio con il pacchetto *express*

```
{
  "name": "hello",
  ...
  "dependencies": {
    "express": "^4.16.3"
  }
  ...
}
```

Per installare tutti i pacchetti che sono sotto *dependencies* nel file *package.json* usiamo il comando *npm install*

Struttura del progetto

In questa sezione vedremo come scrivere le API e come usarle dall'esterno.
La prima best practice è **prendere un approccio a strati** (layered approach) questo è utile per separare i problemi e rendere più facile il debugging (spaghetti code è difficile da gestire e debuggare ad esempio).

In certi framework di NodeJS è possibile fare delle callback functions che vengono eseguite quando un utente richiede la pagina, queste funzioni vengono chiamate **handlers**
È buona norma avere **moduli** differenti per indirizzare differenti "problemi" della nostra applicazione. I diversi moduli avranno la responsabilità di gestire diversi aspetti dell'elaborazione di una richiesta da parte del client
La business logic è la componente del backend che risponde alla richiesta dell'utente, insieme ad una manipolazione di dati, per ritornare una risposta all'utente
Un modulo solitamente è diviso così:

- **Controller**
  Un modulo che definisce l'indirizzamento delle API e gli endpoint, questo livello è accessibile dall'esterno
  In questo modulo possiamo "spacchettare" la richiesta, prendere le informazioni che ci interessano e inviarle al service layer per processarle
- **Service layer**
  Per la logica di business
  Questo modulo contiene **classi** e **metodi**. In questo modulo scriviamo la logica effettiva dell'applicazione
  Questo livello comunica con il database per inviare e ricevere i dati.

- **Data access layer**
  Per comunicare con il database
  Tutte le query, le connessioni al database e i modelli devono essere definiti qui

La seconda best practice è quella di organizzare il nostro progetto in cartelle, questo anche per implementare la prima best practice, quindi possiamo separare i differenti livelli e moduli in diverse **cartelle**.
Questo fornisce chiarezza su **quali** funzionalità sono gestite e **dove** e ci permette di organizzare le nostre classi e metodi in diversi **container** che sono più facili da gestire.

Il nostro progetto sarà strutturato a partire da una cartella source *src*

```
src
    ├── app.js                  app entry point
    ├── /api                controller layer: api routes
    ├── /config                 config settings, env variables
    ├── /services           service layer: business logic
    ├── /models                 data access layer: database models
    ├── /scripts            miscellaneous NPM scripts
    ├── /subscribers        async event handlers
    └── /test               test suites
```

Come prima cosa dobbiamo inizializzare la cartella tramite *npm init*, questo comando creerà anche il file *package.json*, che è il file che "rappresenta" la nostra applicazione.
Nel file *package.json* vengono aggiunti tutte le librerie che ho installato, per questioni di compatibilità
Ogni volta che faccio *npm install* lui va a vedere la lista dei miei pacchetti e mi installa le ultime versioni
Nel file *package.json* dobbiamo aggiungere negli scripts il campo *start* per far capire da dove deve iniziare la mia applicazione:

"*start*": "*node app.js*"

Da ora in poi possiamo scrivere *npm start* per avviare la nostra applicazione.

Dopo mandiamo il comando *npm install*, anche se sembra inutile visto che non ci sono pacchetti da aggiornare/installare, però ci verrà creato il file *package-lock.json* che è una copia di riserva che non si può modificare

Più la nostra applicazione cresce più ci rendiamo conto di quanto servano delle **variabili globali** per far si di condividere certe opzioni e impostazioni con tutti i moduli.
È una best practice salvare tutte queste informazioni in diversi file (in base all'uso) e mettere tutti questi file nella cartella */config*

Ad esempio possiamo creare i file *.env* che hanno le cosiddette **variabili d'ambiente** (che sono le variabili globali). Qui possiamo salvare le credenziali di accesso al database e le chiavi delle API.

Questi file solitamente vengono untracked nei commit perchè contengono dati sensibili.
Possiamo accedere a queste variabili nel nostro codice usando la libreria *dotenv* (quindi dobbiamo eseguirne l'import con require).
Ogni volta che ci da errori di module dobbiamo prima installare il package npm (in questo caso dobbiamo installare dotenv).

È buona norma seguire il seguente ordine:

1. Creare il modello dati
2. Sviluppo backend
3. Sviluppo interfaccia (frontend)

Così da rendersi subito conto di cosa ci serve e non dover cambiare tutto una volta che ci si accorge dei pezzi mancanti.

Esempio

Prendiamo in considerazione le seguenti richieste di API, quindi pianifichiamo che API ci servono e cosa dovranno fare:

- I can create a new tea object and add it to the database
- I can get all the tea from the database
- I can delete all the tea in the database
- I can get a single tea by querying its name
- I can post a comment to a single tea
- I can delete a single tea from the database

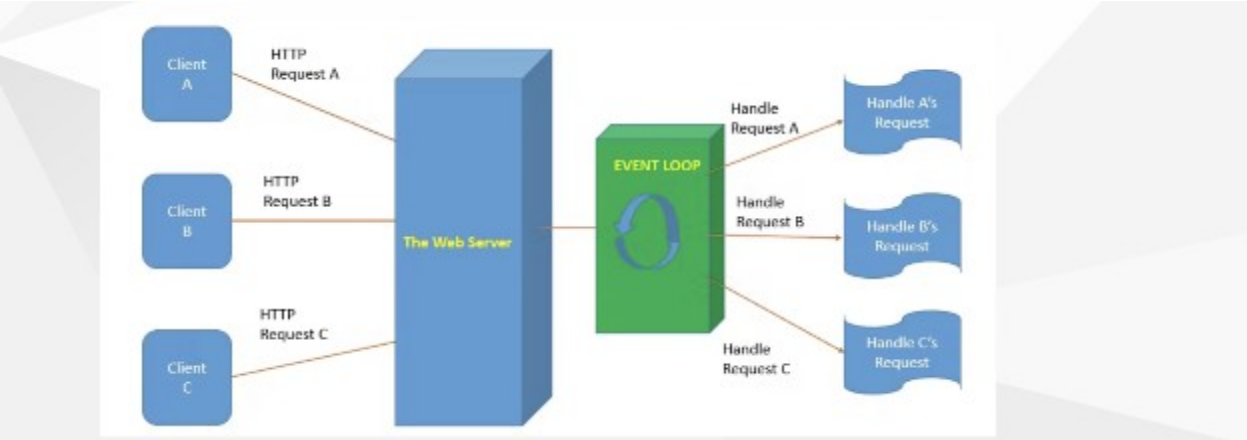
Per ogni riga andremo a scrivere una API diversa.
Come prima cosa dobbiamo definire lo schema del nostro oggetto *tea*, e in base alle richieste possiamo costruire il seguente "prototipo" di oggetto:

```
{
    "name": "Jasmine Tea",
    "image": "an image file url",
    "description": "Jasmine tea is tea scented with the aroma of jasmine blossoms.",
    "keywords": "aromatic, china, sweet",
    "origin":"China",
    "brew time": 2,
    "temperature": 80,
    "comments": []
}
```

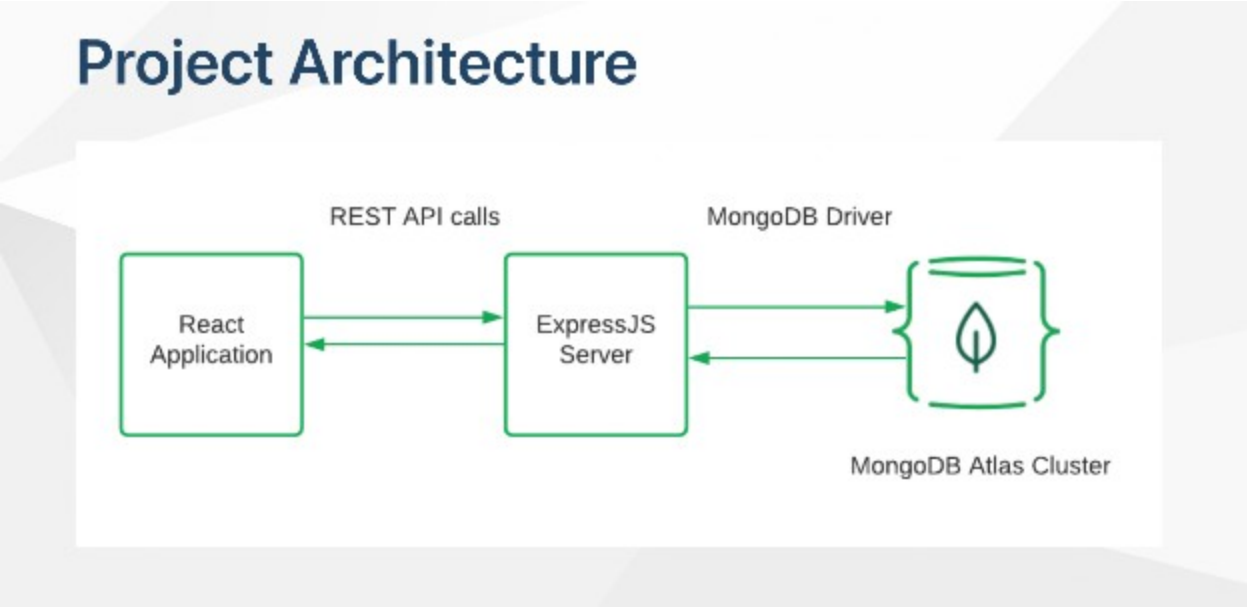
Ora dobbiamo definire la nostra struttura delle API, ovvero scegliere l'indirizzamento (gli URL) e i request methods per ogni API:

Routes	HTTP Methods	Description
/tea	GET	Displays all tea
/tea	POST	Creates a new tea
/tea	DELETE	Deletes all tea
/tea/:name	GET	Displays a specific tea, given its name
/tea/:name	POST	Adds a comment to a specific tea, given its name
/tea/:name	DELETE	Deletes a specific tea, given its name

Node.js ha un modulo HTTP preinstallato che si può usare per fare richieste HTTP e trasferire dati dal server al client. Una chiamata ad un API è una richiesta HTTP ad un URL



Per poter implementare tutti questi meccanismi usiamo il modulo *express*, che è un framework di Node.js che mette a disposizione una serie di funzioni per applicazioni web e mobile.
Express è un linguaggio che ci permette in javascript di interfacciarci con il database.
Il frontend sarà scritto in react e comunicherà con il backend tramite le API



Riscriviamo il nostro web server utilizzando *express*:

```
const express=require('express');
const app=express();


app.get("/", function (req, res) {
  res.send(req.headers, req.originalUrl, req.method, req.body);
});

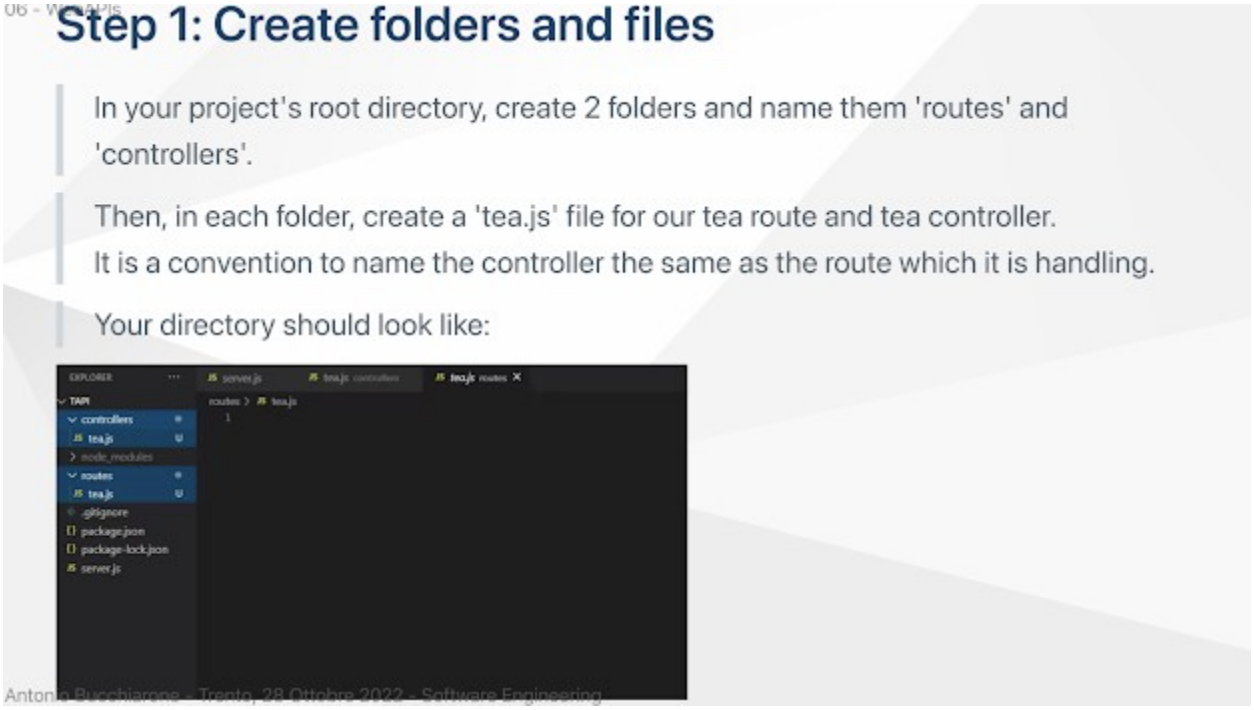
app.listen(3000, () =>
  console.log('Demo app listening on port 3000!'),
);
```

Possiamo costruire ogni API definendo il metodo, l'URL, e l'handler che verrà chiamato:

$$app.METHOD(PATH,\ HANDLER)$$

In questa maniera possiamo focalizzarci sulla logica dei servizi che vogliamo implementare senza preoccuparci del problema delle richieste (ad esempio controllare appena quale è il metodo, controllare quale è l'URL)

Vediamo come implementare tutto questo nel nostro esempio:



Step 2: The First Route and Controller

- Now, open your **routes/tea.js** file. We can create our first route as follows:

1. Create an express router object to set up our routes
2. Import our tea controller from our controllers/tea.js file we created earlier
3. Create our first route with the controller function as the callback to handle the request.
4. Export the route to use in our server.js

```
const express = require('express'); //import express

// 1.
const router  = express.Router();
// 2.
const teaController = require('../controllers/tea');
// 3.
router.post('/tea', teaController.newTea);
// 4.
module.exports = router; // export to use in server.js
```

> At this point, we have not yet created the newTea function but we'll do that right now.

- In controllers/tea.js:

```
// newTea function for post tea route
const newTea = (req, res, next) => {
   res.json({message: "POST new tea"}); // dummy function for now
};

module.exports = {newTea};
```

> Now your first **route** and its **controller** is successfully created!
> Let's add the **routes** to the **server** so that it can access them.

- Our **server.js** is now updated with 2 lines:

```
const routes = require('./routes/tea'); //to import the routes/tea.js
app.use('/', routes); //to use them via express.
```

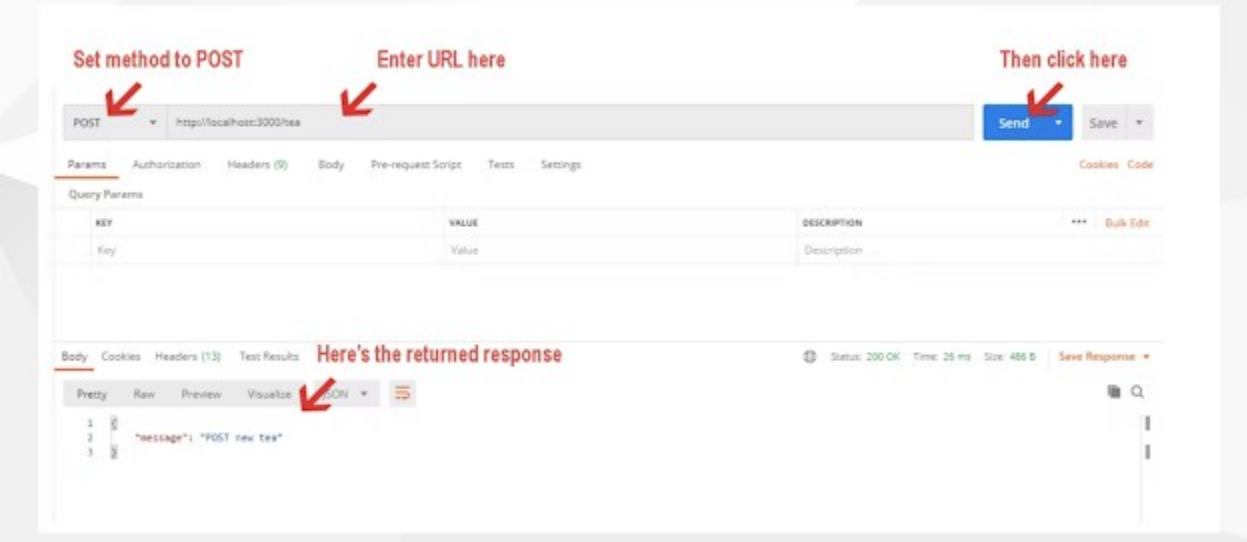
Ecco come possiamo gestire l’oggetto richiesta (come gli header, l’url e i parametri):

```
// Handling GET requests
app.get('/search', function(req, res){
  console.log(util.inspect(req.headers, {showHidden: false, depth: null}))
  console.log(util.inspect(req.url, {showHidden: false, depth: null}))
  console.log(util.inspect(req.query, {showHidden: false, depth: null}))
  res.status(200).send('These are the items found!');
});
// Handling POST requests
app.post('/subscribe', function(req, res){
  console.log(util.inspect(req.headers, {showHidden: false, depth: null}))
  console.log(util.inspect(req.params, {showHidden: false, depth: null}))
  res.status(201).send('You are now subscribed!');
});
```

Ed ecco invece come costruire oggetti di risposta:

```
// text of html
res.send('text')
// json
res.json({ user: 'tobi' })
// status code
res.sendStatus(404)
// chainable status code
res.status(404).sendFile('/absolute/path/to/404.png')
```

Postman serve per verificare che le nostre API siano state fatte correttamente, scegliendo metodo e URL.



Dopo aver scritto tutte le API modificando route e controllers, dobbiamo creare la connessione al DB:

```
npm install --save mongoose
```

- After installing mongoose, add the following to our **server.js** file:

```
//import mongoose
const mongoose = require('mongoose');

//establish connection to database
mongoose.connect(
    'mongodb+srv://<username>:<password>@cluster0.eetsx.mongodb.net/<dbname>',
    { useNewUrlParser: true, useUnifiedTopology: true },
    (err) => {
        if (err) return console.log("Error: ", err);
        console.log("MongoDB Connection -- Ready state is:", mongoose.connection.readyState);
    }
);
```

Per rendere più sicura la nostra connessione al db andiamo a inserire i dati di accesso al database come variabili d'ambiente, ovviamente sostituendo il link in chiaro con la variabile d'ambiente che andiamo a creare

In model andiamo a inserire i schemi di database di mongodb tramite mongoose

```
const mongoose = require("mongoose"); //import mongoose

// tea schema
const TeaSchema = new mongoose.Schema({
    name: {type:String, required:true},
    image: String,
    description: String,
    keywords: String,
    origin: String,
    brew_time: Number,
    temperature: Number,
    comments: [{ text: String, date: {type:String, default: new Date()} }]
});

const Tea = mongoose.model('Tea', TeaSchema); //convert to model named Tea
module.exports = Tea; //export for controller use
```

Let's import our **tea model** we created into the **controllers/tea.js**

```
//import tea model
const Tea = require('../models/tea');
```

La sintassi usata è la seguente:

```
//Syntax
property: {type: SchemaType (i.e. String, Date, Number),
                  other options (i.e. default, required)}

//Examples
name: {type: String, required: true}
description: String   //short for {type: String}
```

Vediamo l'implementazione di *newTEA*

- In this function, we will create a **new tea object** by supplying its key-value pairs to **req.body** and then save it to the database.
- First, we need to be able to parse **form data** with our Express server. We can install the **multer package** with:

```
npm install --save multer
```

- Import multer to our routes/tea.js file:

```
const multer = require('multer');
const upload = multer();
```

- In **controllers/tea.js**:

```
//POST tea
const newTea = (req, res) => {
    //check if the tea name already exists in db
    Tea.findOne({ name: req.body.name }, (err, data) => {

        //if tea not in db, add it
        if (!data) {
            //create a new tea object using the Tea model and req.body
            const newTea = new Tea({
                name:req.body.name,
                image: req.body.image, // placeholder for now
                description: req.body.description,
                keywords: req.body.keywords,
                origin: req.body.origin,
                brew_time: req.body.brew_time,
                temperature: req.body.temperature,
            })

            // save this object to database
            newTea.save((err, data)=>{
                if(err) return res.json({Error: err});
                return res.json(data);
            })
        //if there's an error or the tea is in db, return a message
        }else{
            if(err) return res.json(`Something went wrong, please try again. ${err}`);
            return res.json({message:"Tea already exists"});
        }
    })
};
```

Vediamo ancora qualche API:

```
//GET all teas
const getAllTea = (req, res) => {
    Tea.find({}, (err, data)=>{
        if (err){
            return res.json({Error: err});
        }
        return res.json(data);
    })
};
```

```
//DELETE teas
const deleteAllTea = (req, res) => {
    Tea.deleteMany({}, err => {
        if(err) {
          return res.json({message: "Complete delete failed"});
        }
        return res.json({message: "Complete delete successful"});
    })
};
```

```
const getOneTea = (req, res) => {
    let name = req.params.name; //get the tea name

    //find the specific tea with that name
    Tea.findOne({name:name}, (err, data) => {
    if(err || !data) {
        return res.json({message: "Tea doesn't exist."});
    }
    else return res.json(data); //return the tea object if found
    });
};
```

Ora nella cartella routes andiamo ad inserire tutti i “collegamenti”:

```
const express = require('express');
const router  = express.Router();
const teaController = require('../controllers/tea');

router.get('/tea', teaController.getAllTea);
router.post('/tea', teaController.newTea);
router.delete('/tea', teaController.deleteAllTea);

router.get('/tea/:name', teaController.getOneTea);
router.post('/tea/:name', teaController.newComment);
router.delete('/tea/:name', teaController.deleteOneTea);

module.exports = router;
```

E nella cartella controller avremo:

```
//GET '/tea'
const getAllTea = (req, res, next) => {
    res.json({message: "GET all tea"});
};

//POST '/tea'
const newTea = (req, res, next) => {
    res.json({message: "POST new tea"});
};

//DELETE '/tea'
const deleteAllTea = (req, res, next) => {
    res.json({message: "DELETE all tea"});
};

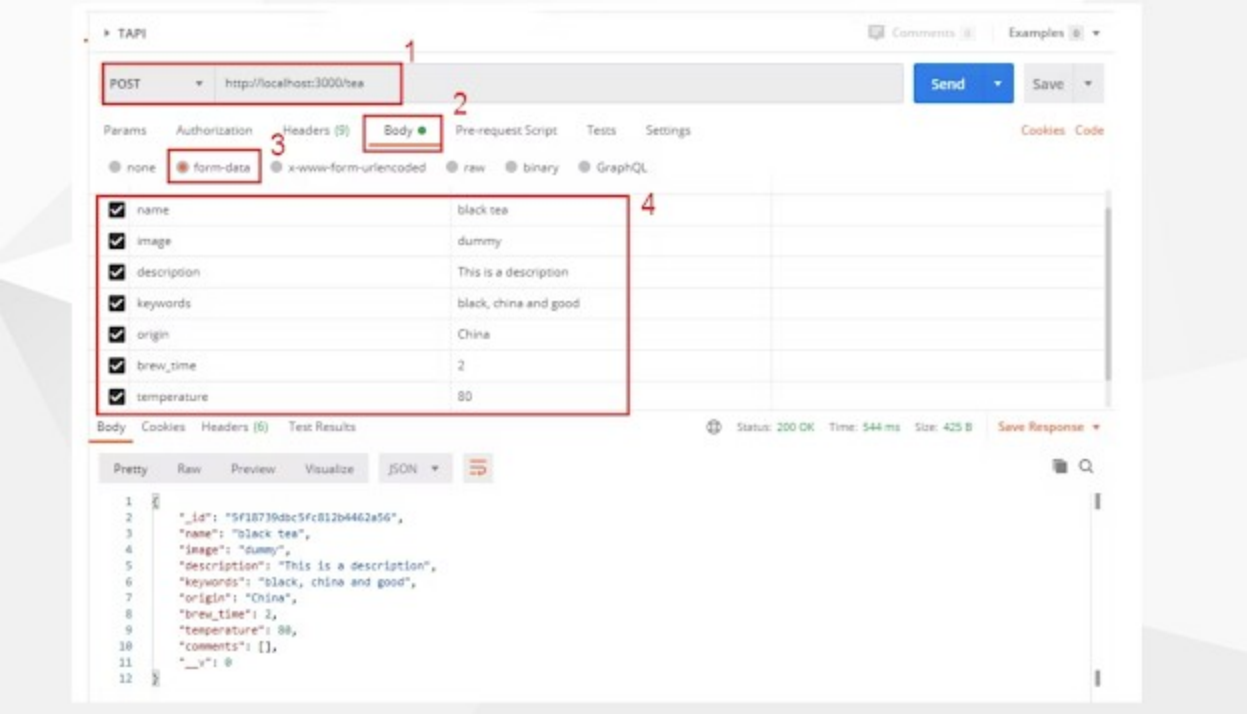
//GET '/tea/:name'
const getOneTea = (req, res, next) => {
    res.json({message: "GET 1 tea"});
};

//POST '/tea/:name'
const newComment = (req, res, next) => {
    res.json({message: "POST 1 tea comment"});
};

//DELETE '/tea/:name'
const deleteOneTea = (req, res, next) => {
    res.json({message: "DELETE 1 tea"});
};

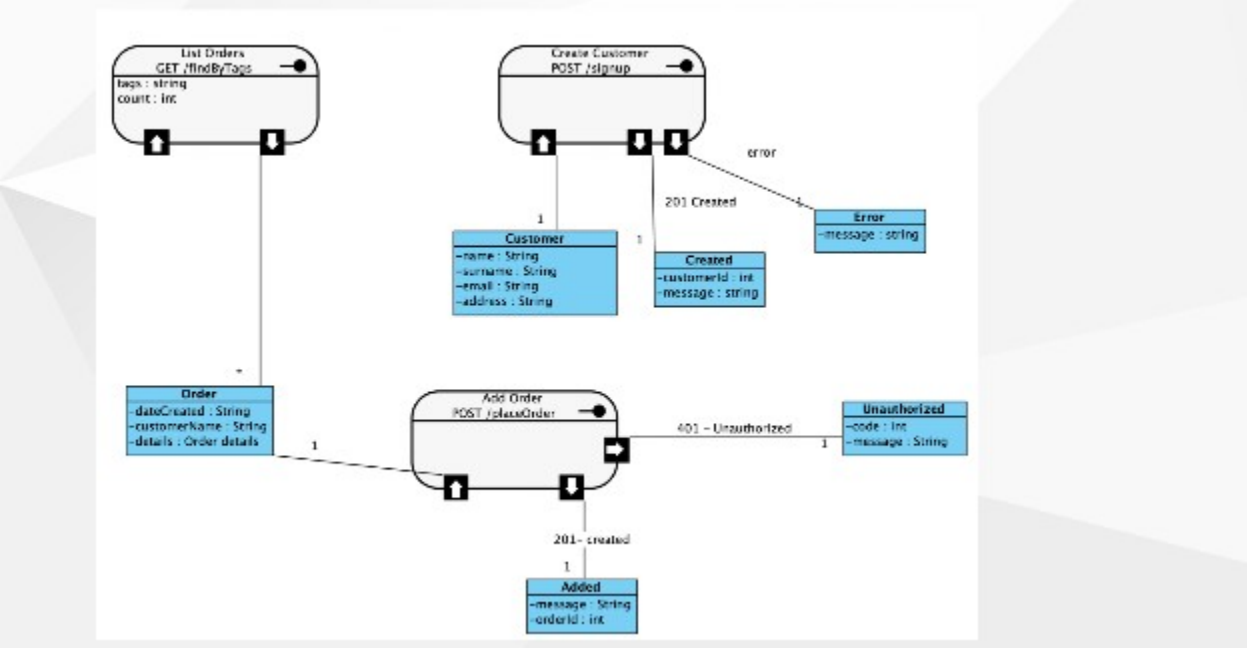
//export controller functions
module.exports = {
    getAllTea,
    newTea,
    deleteAllTea,
    getOneTea,
    newComment,
    deleteOneTea
};
```

Possiamo ora testare su POSTMAN se le API che abbiamo creato funzionano:



Da resource model a implementazione

Supponiamo di aver identificato questo modello di risorse:



Vediamo ora come fare il passo di andare ad implementare *signup*
Come prima cosa dobbiamo:

1. Creare il modello *Customer* per il DB
2. Creare la route *Customer* con la API endpoint (per chiamarla)
3. Creare il controller *Customer* per l'implementazione dell'API (per implementarla logicamente)

Ecco i tre passi:
In *models* / definiamo il modello:

```
const mongoose = require("mongoose"); //import mongoose

// customer schema
const CustomerSchema = new mongoose.Schema({
    name: String,
    surname: String,
    email: String,
    address: String
});

const Customer = mongoose.model('Customer', CustomerSchema); //convert to model named Customer
module.exports = Customer; //export for controller use
```

In *route* / definiamo l'endpoint di questa API:

```
const express = require('express'); //import express

// 1.
const router = express.Router();

const authController = require('../controllers/auth.controller');


router.post('/auth/signup', authController.signup);

// 4.
module.exports = router; // export to use in server.js
```

Ora sviluppiamo il comportamento in *controller* /:

```
if (!customer) {
        const newCustomer = new Customer({
            name: req.body.name,
            surname: req.body.surname,
            email: req.body.email,
            address: req.body.address,
        })


        // save this object to database
        newCustomer.save((err, data) => {
            if (err) return res.json({ Error: err });
            return res.json({ message: "Customer Created", data });
        })

    }
```

Swagger

Swagger è una libreria npm usata per documentare le API (anche per progettare, costruire e usare le API)
Per installarla bisogna usare il comando:

$$npm \ \ i \ \ swagger\text{-}ui\text{-}express$$

La specifica tramite swagger può essere fatta in due formati: YAML o JSON.
Noi useremo JSON e faremo la documentazione tramite file javascript (.js)

Documentare una API significa **spiegare** (in modo formale) come utilizzarla

Express setup `app.js`

```
const express = require('express');
const app = express();
const swaggerUi = require('swagger-ui-express');
const swaggerDocument = require('./swagger.json');

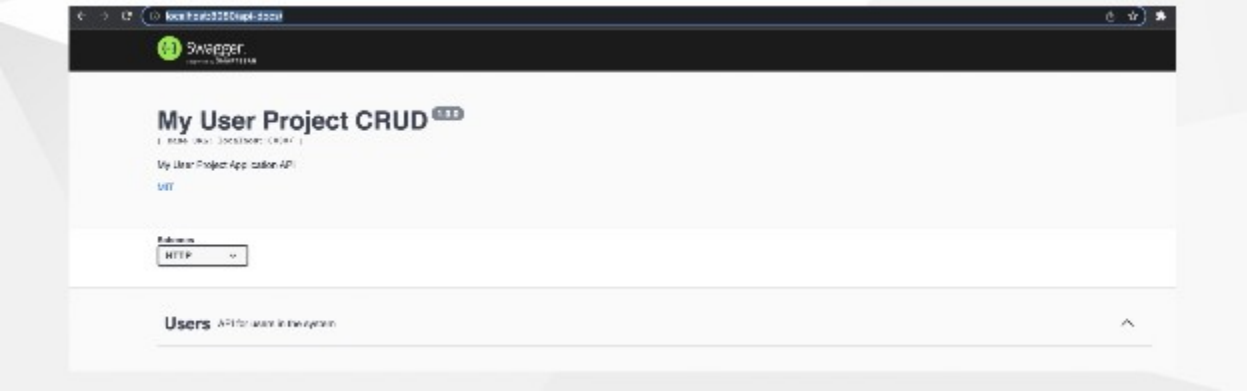
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

Così facendo collego la mia documentazione all'URL */api-docs*, dico di usare *.serve* della libreria swagger e indico di usare il file *swaggerDocument* per la documentazione

Dobbiamo quindi creare il file *swagger.json* nel root del nostro progetto, e come parte iniziale mettiamo le informazioni generali della nostra applicazione:

```
{
  "swagger": "2.0",
  "info": {
    "version": "1.0.0", //version of the OpenAPI Specification
    "title": "My User Project CRUD",
    "description": "My User Project Application API",
    "license": {
      "name": "MIT",
      "url": "https://opensource.org/licenses/MIT"
    }
  },
  "host": "localhost:8000",
  "basePath": "/",
  "tags": [
    {
      "name": "Users",
      "description": "API for users in the system"
    }
  ],
  "schemes": ["http"],
  "consumes": ["application/json"],
  "produces": ["application/json"]
}
```

Già con questo codice saremo in grado di visualizzare l'interfaccia grafica di swagger al link http://localhost:8080/api-docs/



Ora dobbiamo creare la documentazione per ogni nostra API CRUD (Create: POST, Read: GET, Update: PUT, Delete: DELETE)

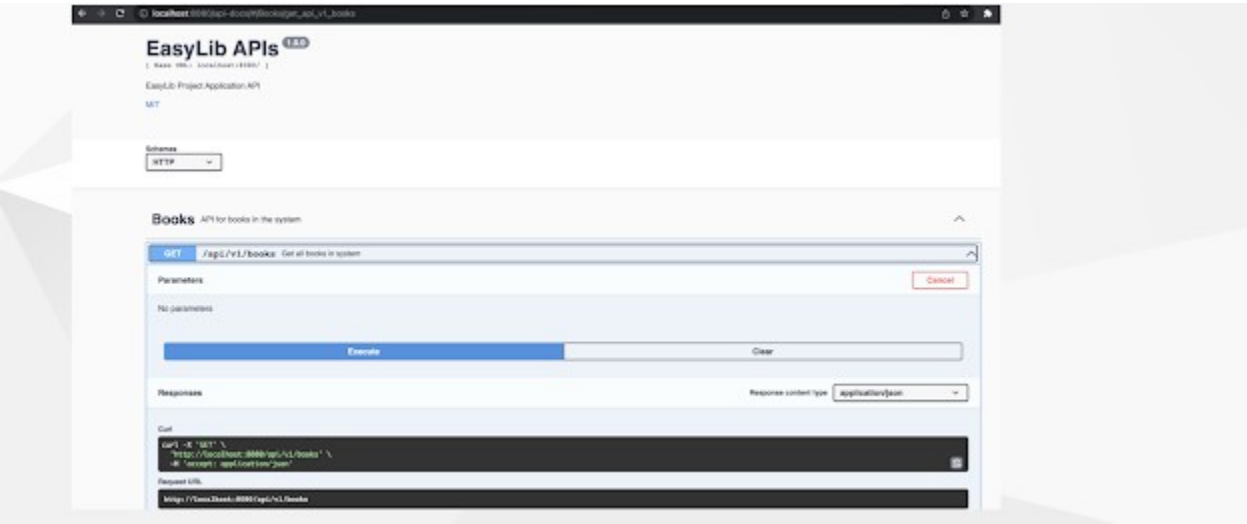
Ecco un esempio di documentazione di API GET:

```
"paths": {
        "/api/v1/books": {
            "get": {
                "tags": [
                    "Books"
                ],
                "summary": "Get all books in system",
                "responses": {
                    "200": {
                        "description": "OK",
                        "schema": {
                            "$ref": "#/definitions/Books"
                        }
                    }
                }
            }
        }
    },
```


Visto che ho usato uno schema come oggetto in risposta, dobbiamo specificarlo:

```
"definitions": {
        "Books": {
            "required": [
                "title"
            ],
            "properties": {
                "title": {
                    "type": "string"
                }
            }
        }
    }
```

Ora vedremo la documentazione di questa API:



Posso anche chiamare le API direttamente dalla UI di swagger e vedere la risposta

Autenticazione

Le “interazioni” con le API sono senza stato, quindi ogni richiesta è assestante e non esiste il concetto di **sessione**.
Di conseguenza, per implementare il concetto di autenticazione dobbiamo usare un approccio differente, detto **token-based**.
I due concetti principali sono:

- Autenticazione: **chi** sei
- Autorizzazione: **cosa** puoi fare

Il **token** viene assegnato al login e ha una durata, dopo la quale non è più valido.

Come prima cosa quindi dobbiamo **ottenere** questo token, ovvero fare l’**autenticazione**:

Send a `POST` request to `/api/authenticate` with `{name: 'admin', password: '123'}` encoded as `x-www-form-urlencoded`.

Poi dobbiamo **inviare** il token per ottenere l’**autorizzazione**:

- Send the token in the **HEADER** parameter `x-access-token`
- You can also send the token as a **URL** parameter: `/api/users?token=YOUR_TOKEN`
- Or you can send the token as a **POST** parameter `token`

Nel nostro progetto dobbiamo scaricare la libreria JWR (ovvero JsonWebToken, che ci permette di implementare questo meccanismo in maniera semplice) con il comando:

npm install jsonwebtoken

Esempio EasyLib

Vediamo la pagina *autenthication.js*, che è la pagina che controlla le credenziali e ritorna il token in caso positivo:

`\app\authentication.js`

```
router.post('', async function(req, res) {
  let user = await Student.findOne({ email: req.body.email }).exec()

  if (!user)                              res.json({success:false,message:'User not found'})
  if (user.password!=req.body.password) res.json({success:false,message:'Wrong password'})

  // user authenticated -> create a token
  var payload = { email: user.email, id: user._id, other_data: encrypted_in_the_token }
  var options = { expiresIn: 86400 } // expires in 24 hours
  var token = jwt.sign(payload, process.env.SUPER_SECRET, options);

  res.json({ success: true, message: 'Enjoy your token!',
    token: token, email: user.email, id: user._id, self: "api/v1/" + user._id
  });
});
```

```
app.use('/api/v1/authentications', authentication);
```

La variabile $process.env.SUPER_SECRET$ usata per creare il token è la chiave segreta dell'applicazione che deve appunto essere nascosta (è la variabile che fa in modo che il token non sia generabile da altre persone).

```
module.exports = {SUPER_SECRET: 'is2laboratory2022'} // .env file
```

Values defined in `.env` are loaded by *dotenv* into `process.env.*`

- `require('dotenv').config()` load within the code
- `node -r dotenv/config your_script.js` -r flag can be used to preload dotenv
- `npm run start_local` (see script defined in package.json)

Vediamo la pagina *tokenChecker.js* che ci permette di verificare se il token posseduto dall'utente è valido:

```
const tokenChecker = function(req, res, next) {
  // header or url parameters or post parameters
  var token = req.body.token || req.query.token || req.headers['x-access-token'];

  if (!token) res.status(401).json({success:false,message:'No token provided.'})

  // decode token, verifies secret and checks expiration
  jwt.verify(token, process.env.SUPER_SECRET, function(err, decoded) {
    if (err) res.status(403).json({success:false,message:'Token not valid'})
    else {
      // if everything is good, save in req object for use in other routes
      req.loggedUser = decoded;
      next();
    }
  });
};
```

Possiamo chiamare questa funzione singolarmente su ogni route su cui richiediamo l'autenticazione, chiamando poi la funzione "vera":

Require authentication only on specified routes

```
app.use('/api/v1/students/me', tokenChecker);  // token validation middleware
app.use('/api/v1/booklendings', tokenChecker); // token validation middleware
// after tokenChecker apply resource routing
app.use('/api/v1/booklendings', booklendings); // resource router middleware
```

Oppure possiamo inserirla senza indirizzo prima di tutte le route su cui richiediamo l'autenticazione:

Position of token validation middleware is important!

```
// Non-protected routes e.g.
app.use('/api/v1/authentications', authentication);
app.use('/api/v1/books', books);

app.use(tokenChecker); // Token validation middleware; Applies on every routes after this point
// Protected routes e.g.
app.use('/api/v1/booklendings', booklendings);
```

FrontEnd

HTML

Le chiamate asincrone dal frontend al backend sono effettuate tramite il metodo *fetch*(), mentre la pagina *html* è aggiornata lavorando direttamente sul **DOM** (Document Object Model) della pagina html.
(Per il progetto è richiesto come minimo avere una pagina *html* e una pagina *javascript* per il frontend)
Ecco un esempio semplicissimo della pagina *html*:

```
<h1>EasyLib</h1>

<form action="api/v1/students" method="post" name="loginform" id="loginform">
  <span>Logged User:</span> <span id="loggedUser"></span>
  <input name="email" value="mario.rossi@unitn.com" id="loginEmail"/>
  <input name="email" value="123" id="loginPassword"/>
  <button type="button" onclick="login()">LogIn</button>
</form>

<h2>Books:</h2> <ul id="books"></ul>

<h2>Insert new book:</h2>
<form action="api/v1/books" method="post" name="bookform" id="bookform">
  <input name="title" value="title" id="bookTitle"/>
  <button type="button" onclick="insertBook()">Insert new book</button>
</form>

<script src="script.js"></script>
```

E la pagina *javascript*:

```
var loggedUser = {} // This variable stores the logged in user

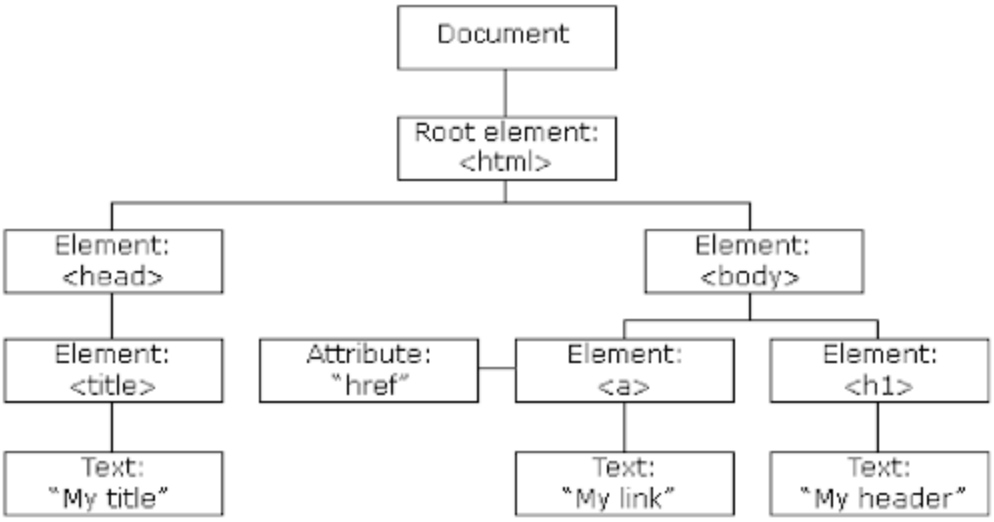
//This function is called when login button is pressed.
function login() { ... }
// This function refresh the list of books
function loadBooks() { ... }
loadBooks();
// This function is called by the Take button beside each book.
function takeBook(bookUrl) { ... }
// This function refresh the list of bookLendings.
function loadLendings() { ... }
// This function is called by clicking on the "insert book" button.
function insertBook() { ...}
```

```
function login() {
  var email = document.getElementById("loginEmail").value;
  var password = document.getElementById("loginPassword").value;

  fetch('../api/v1/authentications', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify( { email: email, password: password } ),
  }).then((resp) => resp.json()) // Transform the data into json

  .then(function(data) { // Here you get the data to modify as you please
    loggedUser.token = data.token;
    loggedUser.email = data.email;
    loggedUser.id = data.id;
    loggedUser.self = data.self;
    document.getElementById("loggedUser").innerHTML = loggedUser.email;
    loadLendings();
    return;
  }).catch( error => console.error(error) );
};
```

La struttura di un HTML DOM (Document Object Model) è la seguente:



Con questo modello, javascript ottiene tutti i “poteri” che gli servono per rendere dinamico HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

Un esempio di cambiamento di contenuto di un elemento è il seguente:

```html
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

Un elemento può essere identificato dal suo *id* o dal suo *name* (gli *id* sono unici, i *name* possono essere assegnati a più elementi):

> Finding HTML Element by Id

```javascript
const element = document.getElementById("intro");
```

> Finding HTML Elements by Tag Name

```javascript
const element = document.getElementsByTagName("p");
```

Ecco un esempio dove controlliamo se un form è “valido” in javascript prima di inviare i dati al backend:

```html
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>

<h2>JavaScript Validation</h2>

<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>

</body>
</html>
```


VueJS

Vue è un framework basato su JavaScript usato per costruire interfacce utente.
È costruito sopra ad HTML, CSS e JavaScript e fornisce una programmazione dichiarativa e basata sui componenti che aiuta a sviluppare interfacce (semplici e complesse).
Permette quindi di fare uno sviluppo del frontend basato sul concetto di componenti.

```
import { createApp } from 'vue'
createApp({
  data() { return { count: 0 } }
}).mount('#app')
```

```
<div id="app">
  <button @click="count++">
    Count is: {{ count }}
  </button>
</div>
```

Vue estende HTML standard con un template sintattico che permette di descrivere dichiarativamente l'output HTML basato sullo stato JavaScript

Vue traccia automaticamente i cambiamenti di stato JavaScript e aggiorna efficientemente il DOM quando i cambiamenti accadono.

Per utilizzare Vue possiamo:

- Usare un **tool**
  Una configurazione di build ci permette di usare Vue Single-File Components (SFCs)
  La build ufficiale di setup di Vue è basato su Vite, un tool di build di frontend
  - Online
    Si può usare Vue con un SFCs online su StackBlitz
  - Locale
    Con *npm init vue@latest* si installa ed esegue *create-vue*, l'ufficiale base di progetto di Vue.
- Senza un tool
  Bisogna semplicemente copiare del codice in un file HTML (per importare Vue e le sue funzionalità)

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>

<div id="app">{{ message }}</div>

<script>
  const { createApp } = Vue

  createApp({
    data() {
      return {
        message: 'Hello Vue!'
      }
    }
  }).mount('#app')
</script>
```

Le componenti di Vue possono essere create in due stili differenti:

With **Options API**, we define a component's logic using an object of options.

```
<script>
export default {
  data() { return { count: 0 } },
  methods: {} ...
```

With **Composition API**, we define a component's logic using imported API functions.

```
<script setup>
import { ref, onMounted } from 'vue'
const count = ref(0) // reactive state
function increment() { count.value++ } ...
```

https://vuejs.org/guide/quick-start.html

Noi procederemo con la creazione tramite build tools.
Copiamo “EasyLibVue” oppure iniziamo da zero eseguendo *npm init vue@latest*.

Per runnare dobbiamo eseguire prima *npm install* e poi *npm run dev*, e l’applicazione sarà sulla porta 3030

Visto che il nostro frontend è in un dominio differente rispetto a dove sono le API (8080), non potremo fare chiamate asincrone dal browser alle API (per la politica Cross-Origin del browser). Per permettere queste chiamate usiamo CORS (Cross-Origin Resource Sharing), che è un meccanismo basato su header HTTP che permette ad un server di indicare qualsiasi origine (dominio, schema, porta) oltre a lui da dove il browser permetterà di caricare risorse.
Ecco come implementare questo meccanismo in NodeJS:

Supporting CORS in Node.js `EasyLib\app\app.js`

```
app.use(function (req, res, next) { // Add headers before the routes are defined
    // Website you wish to allow to connect
    res.setHeader('Access-Control-Allow-Origin', 'http://localhost:3000');
    // Request methods you wish to allow
    res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS, PUT, PATCH, DELETE');
    // Request headers you wish to allow
    res.setHeader('Access-Control-Allow-Headers', 'X-Requested-With,content-type');
    // Set to true if you need the website to include cookies in the requests sent
    // to the API (e.g. in case you use sessions)
    res.setHeader('Access-Control-Allow-Credentials', true);
    // Pass to next layer of middleware
    next();
});
```

Using a module https://expressjs.com/en/resources/middleware/cors.html

```
const cors = require('cors')
app.use(cors())
```

Un’ esempio dello stack Vue.js + Node.js + Express + MongoDB -> https://bezkoder.com/vue-node-express-mongodb-mevn-crud/

La nostra applicazione sarà quindi composta:

- Frontend (Vue.js)
- Backend (Node.js + express)
- Database (MongoDB)

Resoconto di cosa fare per la nostra applicazione:
https://didatticaonline.unitn.it/dol/pluginfile.php/1689235/mod_resource/content/1/%20Application%20Development%20%28FrontEnd-BackEnd%29.pdf

Testing

Esistono tantissimi tipi di testing, ma per farla breve ci sono 3 categorie principali:

- Unit testing
- Integration testing
- UI testing

Per il **D4** vedremo solo gli **unit testing**.

Jest è un framework di testing javascript basato sulla semplicità. Jest include sia un **test-runner** che le funzioni **assertion** in un unico package.
In sintesi è una libreria JavaScript per creare, runnare e strutturare i **test**.
Jest è un package NPM ed è uno dei più popolari test runner (scelta di default per i progetti React)

La domanda ora è come faccio a sapere cosa devo testare.
Se si sta scrivendo un'applicazione web un buon punto di partenza sarebbe quello di testare **ogni singola pagina** e ogni possibile **interazione dell'utente**.
Ma le applicazioni web sono fatte anche di codice come **funzioni** e **moduli**, e ovviamente devono essere testati anche loro.

Un test flow tipicamente è strutturato nella seguente maniera:

1. Importare la funzione da testare
2. Dare un input alla funzione (input)
3. Definire cosa ci si aspetta in output (expected output)
4. Controllare se la funzione produce l'output atteso (asser the result)

Per configurare Jest dobbiamo definire uno script nel file $package.json$ del nostro progetto:

```
{
    "scripts": {
        "test": "jest  --coverage"
    }
}
```

> The provided configuration will run the jest command on all files with the extension .test.js
> The --coverage flag will produce a coverage report in the output

Come prima cosa per definire un test dobbiamo scrivere il metodo *describe*, che è un metodo Jest che contiene uno o più test correlati. Ogni volta che iniziamo a scrivere una nuova suite di test dobbiamo raggrupparli in un blocco describe.
describe prende due argomenti, una stringa per descrivere la test suite e una callback function per il wrapping dei test.

Ogni test inizia con la funzione *test*(), che ha due parametri richiesti e uno opzionale:

- Una stringa che descrive la funzionalità che viene testata
- Una callback function che contiene la logica del test da eseguire
- (OPZIONALE) Un timeout in millisecondi (Jest dovrà attendere la fine di questo timeout per completare il test).

Un test si basa su input, funzione e output atteso. Quindi possiamo definire input e output atteso come variabili, e poi per effettuare concretamente il test usare la funzione *expect*, che prende in input la funzione, e confrontare con l'output atteso.
Ecco un esempio:

```
describe("Filter function", () => {
test("it should filter by a search term (link)", () => {
  const input = [
    { id: 1, url: "https://www.url1.dev" },
    { id: 2, url: "https://www.url2.dev" },
    { id: 3, url: "https://www.link3.dev" }
  ];

  const output = [{ id: 3, url: "https://www.link3.dev" }];

  expect(filterByTerm(input, "link")).toEqual(output);

});
});
```

La cosa più corretta da fare è scrivere le funzioni di test in un file separato rispetto al quale dove la funziona da testare vive.
Possiamo usare più funzioni *expect* se sono corretti più output per un solo input.

Vediamo ora un esempio di come costruire un test semplice da zero:
Come prima cosa dobbiamo installare *Jest* con il comando

npm install --save-dev jest

Poi scriviamo la funzione da testare nel file $\sum.js$

```
function sum(a, b) {
  return a + b;
}
module.exports = sum;
```

Poi creiamo il file $\sum.test.js$ dove andiamo a definire i test

```
const sum = require('./sum');

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```

Ora possiamo runnare il test con il comando $npm\ \ test$ e l'output sarà il seguente:

```
PASS  ./sum.test.js
✓ adds 1 + 2 to equal 3 (5ms)
```

Possiamo anche testare le API, con l'utilizzo di un'altra libreria, *fetch*.
Installiamo la libreria con il comando

$$npm\ \ install\ \ --save-dev\ \ node-fetch$$

E poi possiamo richiamare le API e confrontarne la risposta:

```
const fetch = require("node-fetch");
const url = process.env.HEROKU || "http://localhost:3000"
it('works with get', async () => {
    expect.assertions(1)
    expect( ( await fetch(url) ).status ).toEqual(200)
})
it('works with post', async () => {
    expect.assertions(1)
    var response = await fetch(url+'/courses', {
        method: 'POST', body: JSON.stringify({name: 'hello course'}),
        headers: { 'Content-Type': 'application/json' }
    })
    expect( ( await response.json() ).status ).toEqual(201)
})
```

Ecco un esempio di test con token:

Testing EasyLib *token-authenticated* APIs on *mongodb*

`EasyLib\app\booklendings.test.js`

```
const request  = require('supertest');    const app      = require('./app');
const jwt      = require('jsonwebtoken'); const mongoose = require('mongoose');

describe('GET /api/v1/booklendings', () => {

  beforeAll( async () => { jest.setTimeout(8000);
    app.locals.db = await  mongoose.connect(process.env.DB_URL); });
  afterAll( () => { mongoose.connection.close(true); });

  var token = jwt.sign( {email: 'John@mail.com'},
    process.env.SUPER_SECRET, {expiresIn: 86400} ); // create a valid token

  test('POST /api/v1/booklendings with Student not specified', () => {
    return request(app).post('/api/v1/booklendings')
    .set('x-access-token', token).set('Accept', 'application/json')
    .expect(400, { error: 'Student not specified' });
  });
```

Per eseguire richieste API nei nostri test Jest, ci serve un modulo che effettuerà le richieste al nostro endpoint e ci ritornerà la **risposta**. Il modulo è **SuperTest** e si installa con il comando:

npm install --save-dev supertest

Vediamo subito un esempio di come funziona.
Basiamoci su una API che ritorna un array di oggetti come la seguente:

- Start with the response provided by the endpoint http://localhost:3000/space/destinations. It is a simple array of objects representing the destinations space flights will travel to.

```
[
  "Mars",
  "Moon",
  "Earth",
  "Mercury",
  "Venus",
  "Jupiter"
]
```

Ecco come possiamo scrivere il test:

```
const request = require('supertest');
const app = require("../app");

describe('Space test suite', () => {
    test('tests /destinations endpoints', async() => {
        const response = await request(app).get("/space/destinations");
        expect(response.body).toEqual(["Mars", "Moon", "Earth", "Mercury", "Venus", "Jupiter"]);
        expect(response.body).toHaveLength(6);
        expect(response.statusCode).toBe(200);
        // Testing a single element in the array
        expect(response.body).toEqual(expect.arrayContaining(['Earth']));

    });

    // Insert other tests below this line

    // Insert other tests above this line
});
```

Vediamo un esempio con un API un po' più complessa come questa:



Ecco come possiamo scrivere il test:



```
test('tests /space/flights/seats endpoint - starship', async () => {
    const response = await request(app).get("/space/flights/seats");
    expect(response.body.starship).toEqual(expect.arrayContaining([expect.any(Object)]));
    // Checking that the starship Object contains firstClass Object which then contains a set of objects
    expect(response.body.starship).toEqual(expect.arrayContaining(
        [expect.objectContaining({ firstClass: expect.any(Object) })]));

    expect(response.body.starship).toEqual(expect.arrayContaining([expect.objectContaining(
        { businessClass: expect.any(Object) })]));

    // Checking that under the bussinessClass Object we have the array drinks served
    expect(response.body.starship)
        .toEqual(expect.arrayContaining([expect.objectContaining({
            businessClass: expect.objectContaining({ drinksServed: expect.any(Array) })
        })]));

    // Checking that under the firstClass: Object we have the option ludacris in the seatHover Object
    expect(response.body.starship)
    .toEqual(expect.arrayContaining([expect.objectContaining({
        firstClass: expect.objectContaining({ seatHover: expect.objectContaining({
            cryoMode : expect.arrayContaining(['ludacris'])}) })
    })]));

    // Checking that under the firstClass: Object we have the option plaid in the seatHover Object
    expect(response.body.starship)
    .toEqual(expect.arrayContaining([expect.objectContaining({
        firstClass: expect.objectContaining({ seatHover: expect.objectContaining({
            staticMode : expect.arrayContaining(['plaid'])}) })
    })]));
});
```

Deployment

Eseguire il deployment vuol dire trasferire l'applicazione da locale a un server, così che possa venir usata da tutti gli utenti.
CI/CD (Continous Integration / Continuous Deployment) è una pratica dell'ingegneria del software che aiuta i team a lavorare meglio e migliorare il prodotto finale.
Con **GitHub Actions** si può facilmente integrare questa pratica nel proprio progetto GitHub senza l'uso di piattaforme esterne.

Vediamo meglio in cosa consiste questa pratica:

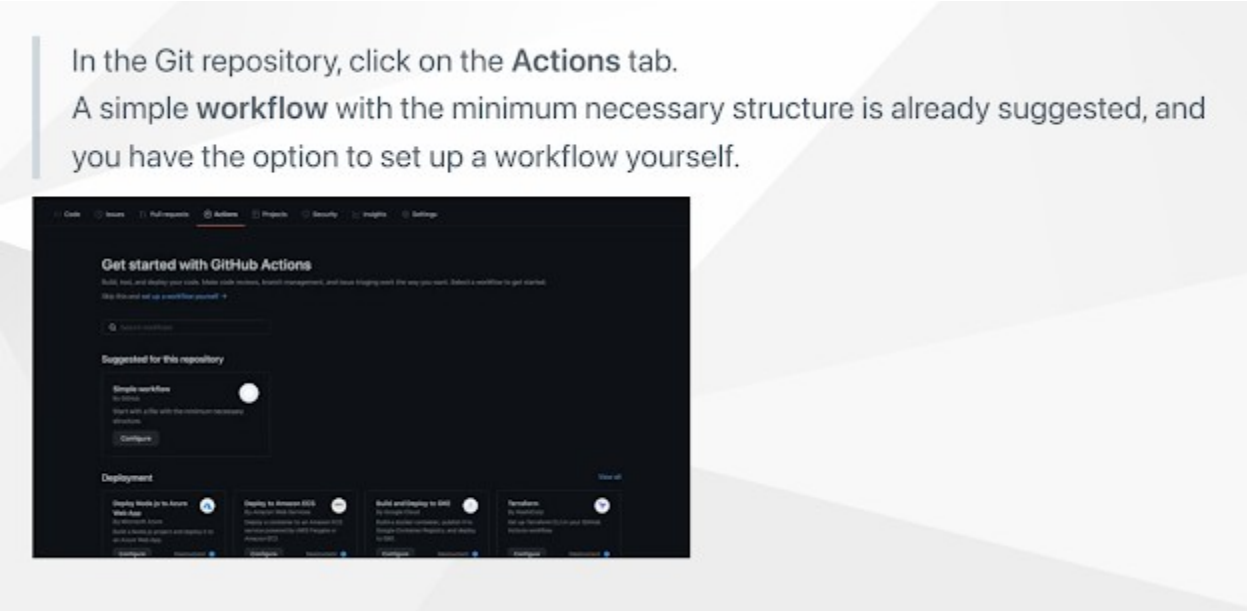
- **Continous integration** (CI)
  È la pratica dell'ingegneria del software che richiede **commit frequenti** in una cartella condivisa.
  Prima della CI, le persona lavoravano su feature branch per settimane o mesi per poi fare il merge sul main branch (e spesso le cose andavano male in questo merge per conflitti e test falliti)
  Con la CI si prevengono questi problemi, incoraggiando piccoli e frequenti aggiornamenti del codice.
  Ogni volta che il codice viene committato su una repository, può essere buildato e testato rispetto ai workflows di installazione per essere sicuri che il codice non introduca errori.
- **Continous deployment** (CD)
  Oltre alla CI, i cambiamenti di codice fatti vengono automaticamente deployati/rilasciati.
  È spesso associato al continous delivery, con la differenza che nella continous delivery deve esserci un "intervento umano", mentre nel continous deployment no (definiamo degli script che ci permettono di fare queste cose in maniera automatica)

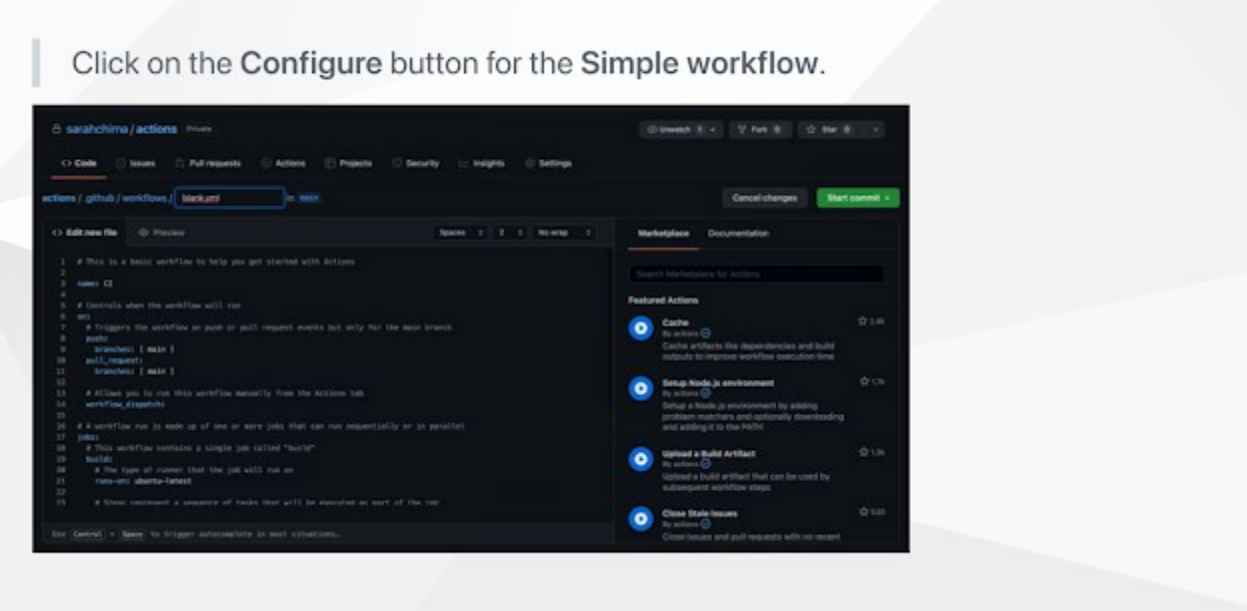
I vantaggi del CI/CD (oltre a quelli già menzionati) sono:

- L'**isolamento dei problemi** rende tutto più semplice e facile.
  Visto che i cambiamenti sono piccoli, è molto più facile individuare quale pezzo di codice ha causato un bug durante il deployment. Questo rende molto facile fixare o fare un rollback quando c'è un bug.
- Il **tempo di revisione del codice** è minore (frequenti e piccoli cambiamenti)
- Ci sono dei **test automatizzati dei flow critici** per il progetto. Questo rende più facile l'impedire delle modifiche che possono "rompere" questi flow.
- Una miglior **qualità del codice** è assicurata.

In GitHub ci sono le **azioni** che permettono di implementare la logica CI/CD, permettendo di automatizzare la build, il test e il deployment del progetto.
Si possono creare dei **workflows** che fanno la build e il test ad ogni pull sulla repository oppure che fanno il deploy.
Si possono impostare delle CI/CD pipeline che vengono eseguite quando una certa azione viene fatta sulla repository.
Si può decidere di eseguire i test su ogni pull request e si può automaticamente deployare le merged pull request

Ecco come aggiungere una azione su una repository github:

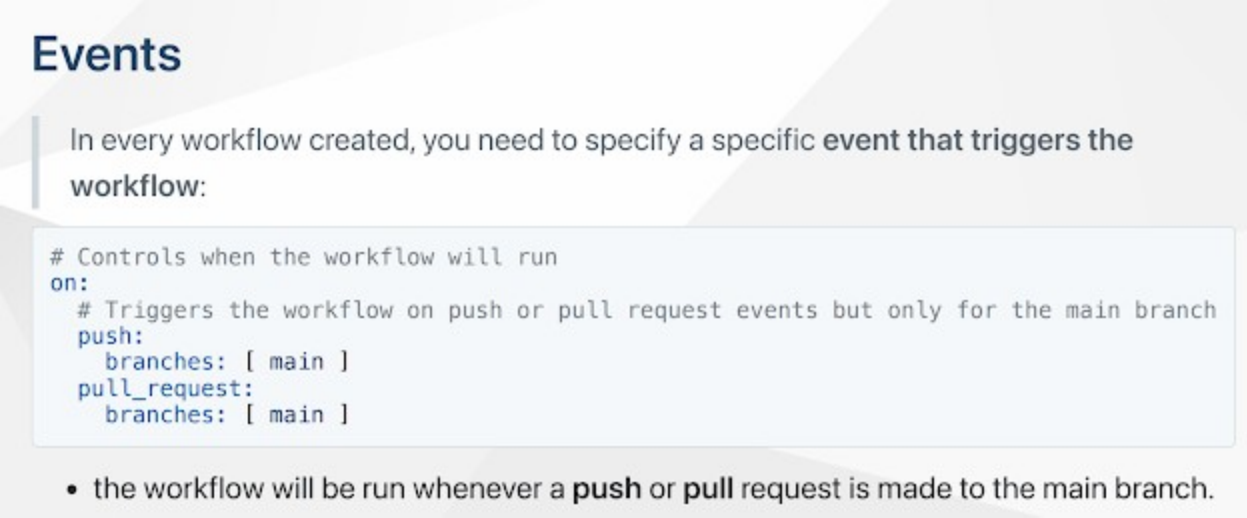




Un **workflow** è un file (.yml) che viene creato quando noi creiamo una azione su github nella cartella *workflows*:

- È un processo automatizzato configurabile che esegue uno o più “lavori”
- È un YAML file
- È contenuto in un YAML file nella cartella workflows è attivato da un evento definito nel file

Gli **eventi** che fanno scattare il workflows vengono definiti nella seguente maniera:

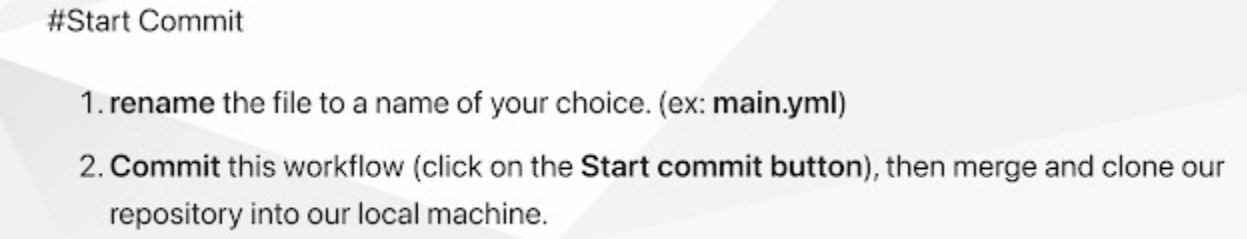


Un **job** è una serie di step che un workflow deve eseguire, gli step vengono eseguiti in ordine e sono dipendenti uno dall’altro.

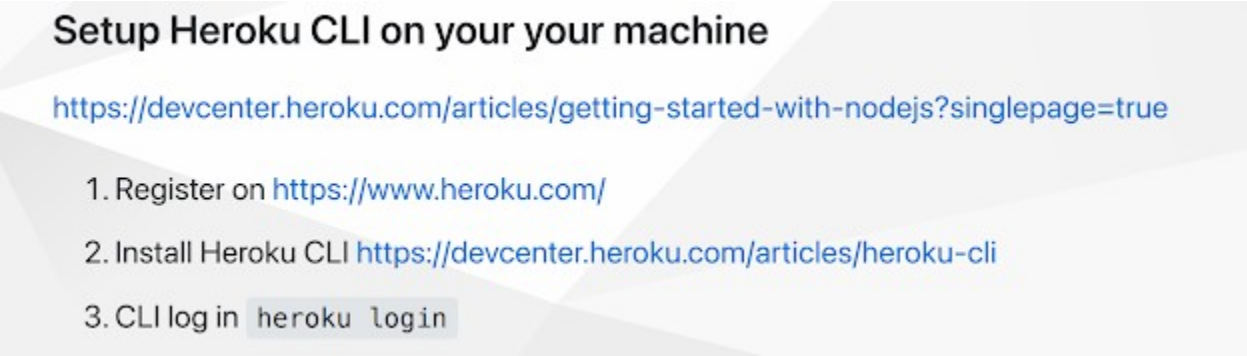
Un **runner** indica il server su cui sarà eseguito il lavoro (Linux, Windows, MacOS):

```
# The type of runner that the job will run on
    runs-on: ubuntu-latest
```

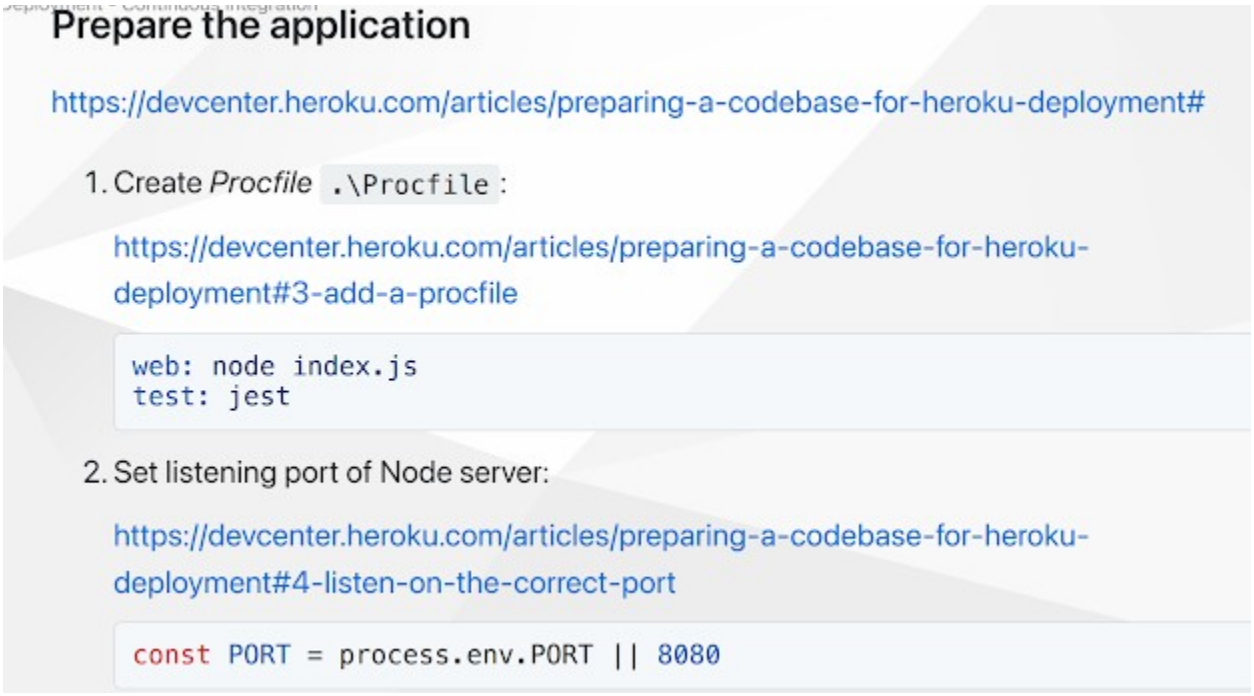
Ritornando sulle azioni di github, possiamo scriverene una da zero oppure usare quelle “predefinite” di github.
Ad esempio per il nostro progetto possiamo usare la azione predefinita $Node.js$ per buildare la nostra applicazione, e quella di Heroku per deployarla.



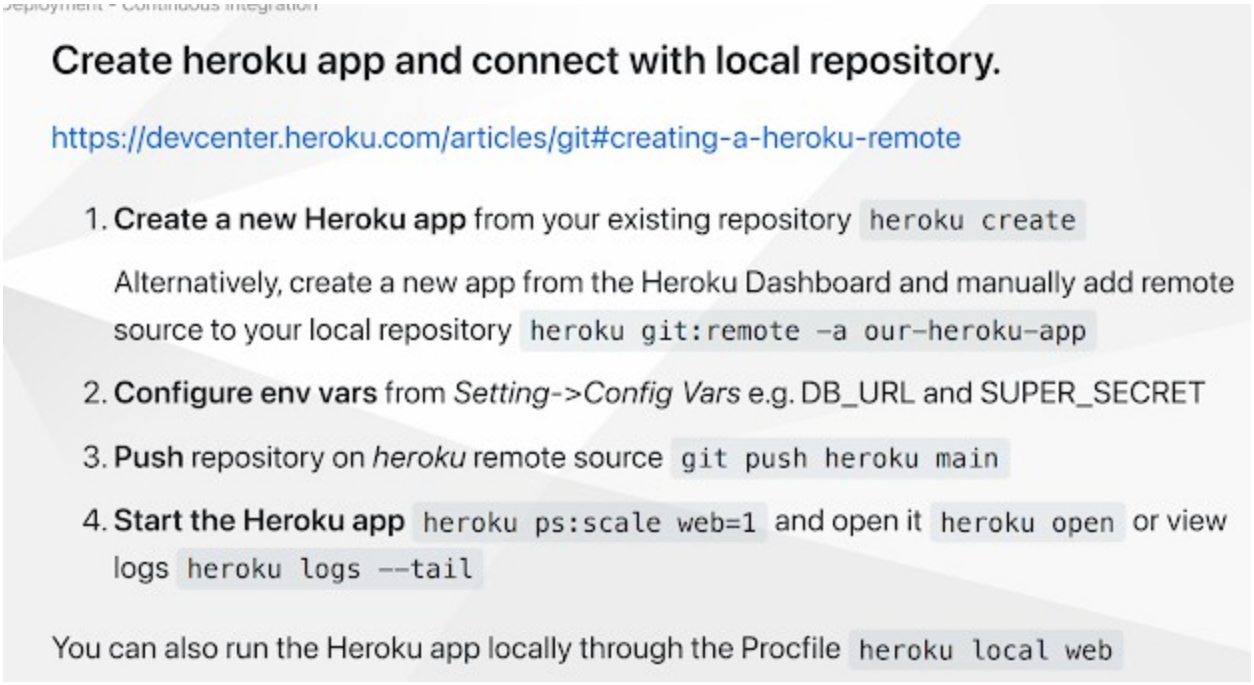
Per fare il deployment, devo salvare la mia applicazione sul cloud, per esempio usando Heroku.



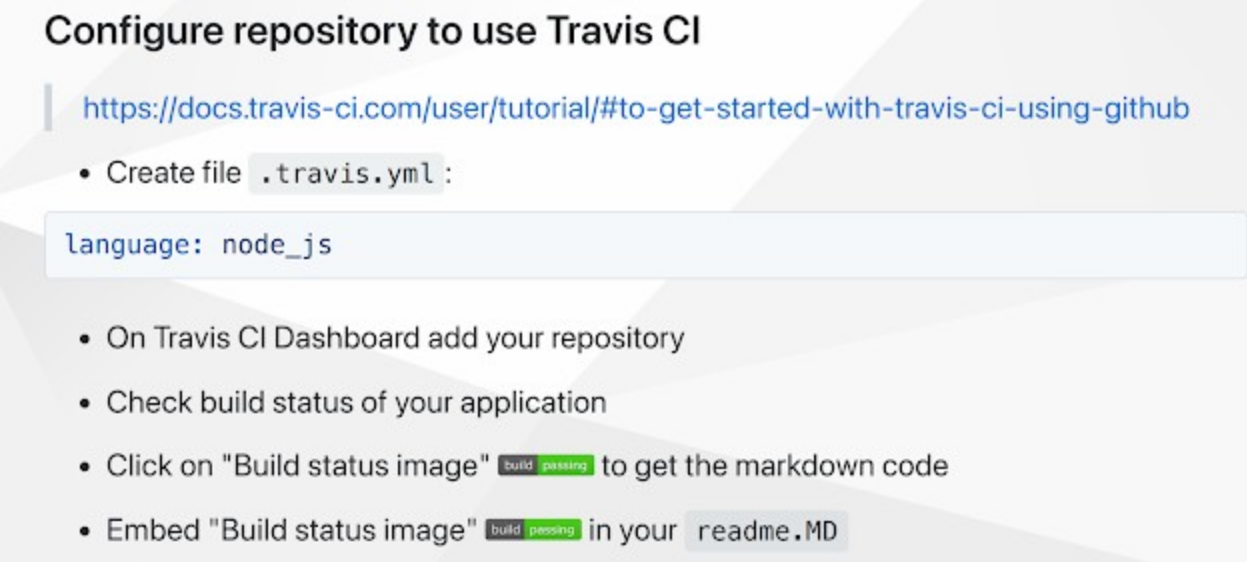
https://devcenter.heroku.com/articles/getting-started-with-nodejs?singlepage=true



https://devcenter.heroku.com/articles/preparing-a-codebase-for-heroku-deployment#

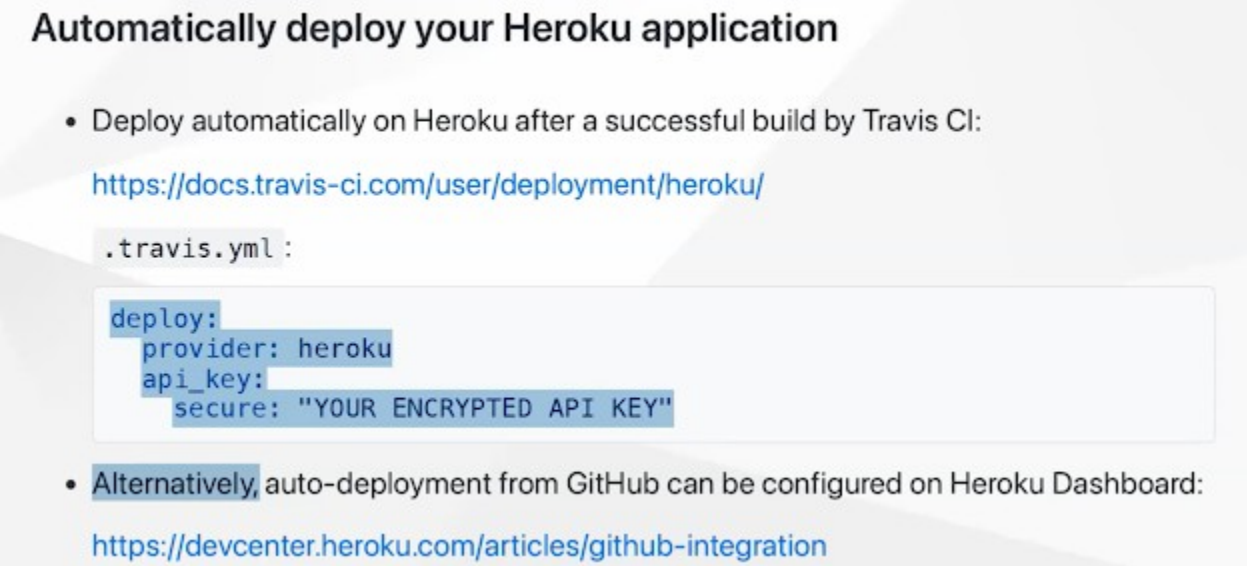


Oppure si può usare Travis CI:



https://docs.travis-ci.com/user/tutorial/#to-get-started-with-travis-ci-using-github

Tramite Travis si può comunque impostare Heroku come provider



È importante ricordarsi di inserire il file *.gitignore*

`.gitignore` - Ignoring files from git versioning

- You can start from generic `.gitignore` file generated on www.gitignore.io, such as, https://www.gitignore.io/api/node,windows,linux,visualstudiocode
- **Make sure to always ingore**: `node_modules` `coverage` `.env`
- Put the `.gitignore` file itself under version control `git add .gitignore`